TraceProv: All You Need is Log - Efficient Bolt-on Provenance Capture for Complex Relational Queries

Vinayak Jha
University of Illinois, Chicago
vinayakj@uic.edu

Boris Glavic
University of Illinois, Chicago
bglavic@uic.edu

ABSTRACT

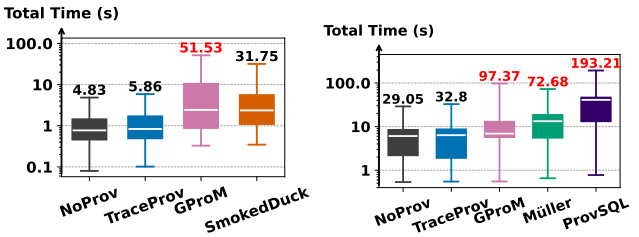
A long line of work has investigated how to add support for provenance capture to relational databases. However, achieving high performance for complex queries has been challenging: systems that have low overhead modify the execution engine of the database while "bolt-on" systems have high overhead. In this work, we demonstrate that, with the right combination of algorithmic and systems techniques, it is possible to achieve competitive performance relying exclusively on database extensibility mechanisms. Our approach is based on a lightweight parallelism-friendly logging framework using memory-mapped files and a two-phase capture strategy that opportunistically combines logging with propagation. We implement our ideas in a system called TraceProv on top of PostgreSQL and DuckDB. TraceProv supports complex SQL queries, utilizes intra-query parallelism, can handle larger-than-memory data, and significantly outperforms SOTA baselines.

PVLDB Reference Format:
Vinayak Jha and Boris Glavic. TraceProv: All You Need is Log - Efficient Bolt-on Provenance Capture for Complex Relational Queries. PVLDB, 14(1): XXX-XXX, 2026.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/realVinayak/traceprov

1 INTRODUCTION

While provenance enables a wide range of use cases [13], adding support for provenance capture to a relational database system is a highly non-trivial task. Existing approaches fall into one of two categories: *Bolt-on* systems are built on top of existing DBMS and express provenance capture computations as SQL code, optionally using UDFs and other extensibility mechanisms [3, 14, 33, 34]. *Native* systems extend the database's execution engine to track provenance [22, 32]. Bolt-on systems are easier to port to new database systems or newer versions of an existing system, but have to work around limitations of SQL and of the database's extensibility interface. Native systems typically modify operator implementations and have full control the representation and storage of provenance and the data flow in a capture computation. Thus, this type of systems has achieved superior performance [22, 32]. However, the native approach requires significant development effort and modifications to the DBMS, reducing portability and maintainability.



(a) DuckDB (b) PostgreSQL

Figure 1: Query runtime distribution for TPC-H SF10. TraceProv is the fastest system on both PostgreSQL & DuckDB.

In this work, we present TraceProv which achieves performance better than native systems using a bolt-on approach using a combination of algorithmic and systems techniques based on a careful analysis of bottlenecks of SOTA methods. Figure 1 shows the distribution of runtimes for provenance capture for the 22 TPC-H [35] queries over a SF10 instance. Systems with red maximum runtimes timed out on some queries or do not support all TPC-H queries. TraceProv exhibits low overhead over executing queries without provenance capture (*NoProv*) and outperforms existing provenance systems on both PostgreSQL and DuckDB, including SmokedDuck [22, 24], a native system.

Example 1 (Running Example). *Consider query Q_1 shown below. Figure 2 shows example instances of the two input tables and the intermediate results of the query expressed in relational algebra (right). The join produces 5 rows belonging to two groups, out of which only one ($a = \alpha_1$) passes the* `HAVING` *condition. The provenance of this single result tuple according to the* lineage *[6, 13] provenance model (shown on the bottom left Figure 2) identifies which input tuples in R_1 and R_2 contributed to this result. These are tuples from R_1 with the group-by attribute value ($a = a_1$) and their join partners in R_2.*

```
Q1: SELECT R1.a, count(*)
    FROM R1 JOIN R2 ON rid = R2.b
    GROUP BY R1.a HAVING count(*) > 2;
```

To capture lineage for an SQL query, it is necessary to track data dependencies between output and input tuples of the query's operators. SOTA systems differ in their dependency tracking strategy.

Annotation propagation vs. logging. *Annotation propagation* systems annotate input rows with unique identifiers and propagate and combine these annotations during query processing such that each (intermediate) result is annotated with its provenance. This makes annotation theoretically effective at retrieving provenance for a

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

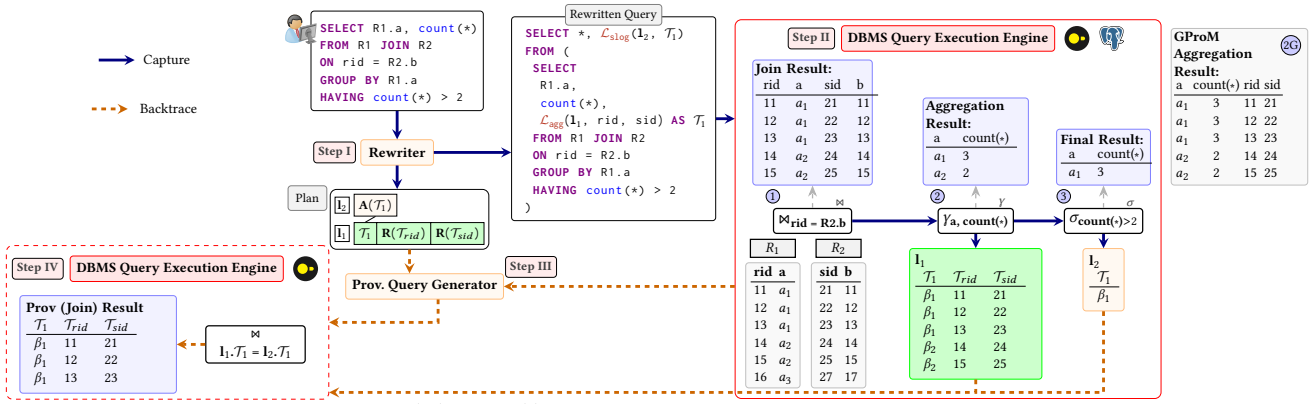


Figure 2: Capturing lineage for $Q_1 : \sigma(\gamma(R_1 \bowtie R_2))$ with TraceProv. (I) instrumenting the query to log (using UDFs (red)) and propagate provenance; (II) execute the instrumented query, logging provenance in two logs; (III) generate an SQL query that joins these logs; (IV) execute this query, returning lineage (bottom left).

result tuple since the provenance is stored inline. However, such annotations are typically variable sized because an output tuple may depend on many input tuples, requiring non-1NF representations [31] or duplication of result rows to distribute provenance across these copies [3, 14]. This increases the intermediate result size (either the number of rows or the size of individual rows) compared to vanilla execution, and significantly degrades performance. The intermediate result size blowup occurs for operators where single result row depends on multiple input rows (e.g, aggregation), and when the provenance of a row is shared, e.g., an uncorrelated nested subquery's provenance is shared across all result tuples produced by the outer query. In the running example in Figure 2, the intermediate result size after (2) consists of two rows. However, GProM [3] will emit 5 rows for the same aggregate (shown in 2G). Thus, successive operators, e.g., (3), will consume more rows, increasing latency.

The alternative to annotation propagation is to *log* sufficient information during query execution and then reconstruct provenance from these logs. In the running example, for the same aggregate (2), TraceProv outputs the original number of rows (2). TraceProv does so by storing the join-result to group mapping in a log $\mathbf{l}_1$. The lineage of an output tuple of the aggregate can then be extracted from $\mathbf{l}_1$. Thus, creating logs avoids the intermediate result size blowup, at the cost of having to write logs and having to joins these logs to reconstruct provenance. TraceProv combines the advantages of propagation with logging: we propagate provenance through operators and only log at operators for which propagation would result in an increase of result size (e.g., aggregation, nested subqueries). Existing systems such as Müller and ProvSQL for PostgreSQL create similar logs via UDA and UDF in database tables and mmaped files, respectively. SmokedDuck [22] modifies the execution engine of DuckDB to log mappings between output and input rows of physical operators as columnar in-memory data structures.

Instrumenting Queries for Capture. In Step I in Figure 2, the input SQL query is instrumented to propagate and log provenance information. TraceProv performs logging via UDAs and UDFs ($\mathcal{L}_{\text{agg}}$ and $\mathcal{L}_{\text{slog}}$). In contrast to systems like GProM which may significantly alter the structure of the input query leading to inefficient query plans [28, 29], TraceProv in most cases preserves the original query structure. Importantly, TraceProv does not alter the execution engine of the database system and, thus, has high portability. In fact, the initial porting of TraceProv from PostgreSQL to DuckDB took less than a week.

Efficient, parallel, and append-only logging. To achieve competitive logging performance to SmokedDuck, TraceProv provides an efficient logging infrastructure for logs backed by mmaped files or in-memory pages. We strictly perform append-only logging and memory accesses during logging. The logical log can be partitioned into multiple physical logs, allowing parallel workers to write into same logical log without blocking. We further maintain zonemaps to optimize reconstruction of provenance from logs.

Efficient Backtrace. After the logging, TraceProv constructs a SQL query joining the logs to backtrace provenance one step at a time (Step III). This query is then executed using DuckDB (Step IV). TraceProv by default computes lineage for all result rows, but also supports tracking a single result and computing provenance polynomials for SPJU queries. Logs are accessed using custom TableFunctions (A TableFunction in DuckDB serves the purpose of a *foreign data wrapper* in SQL/MED). For our running example, the constructed query for reconstructing lineage is a join: $\mathbf{l}_2 \bowtie \mathbf{l}_1$. Like SmokedDuck [22], we reconstruct provenance through backtracing starting from the final query result. SmokedDuck constructs in-memory index structures (using C++ STL data structures) to efficiently serving multiple provenance requests for the same query. While this reduces backtrace time, the index building cost is too high to be amortize in most scenarios. TraceProv instead builds only lightweight data structures for partitioning pruning and data skipping with low logging overhead.

Contributions. In this work, we present `traceprov`, a new provenance system that is competitive with native systems like SmokedDuck [22, 23] and has the portability and bolt-on characteristics of approaches like ProvSQL [33, 34], GProM [3], DataProv [31], and Müller [9, 26]. This work makes the following contributions:

- We present a light-weight, append-only, parallel logging framework for provenance using mmap (in-memory or

backed by files) that tracks sufficient statistics for partition pruning and data skipping during backtrace.
- DBMS-specific wrappers for this framework are used to instrument queries for writing logs and propagating row identifiers according, preserving in most cases the structure of the input query. Provenance is efficiently reconstructed top-down from logs using SQL queries.
- We present an extensive experimental evaluation of TraceProv on DuckDB and PostgreSQL, comparing against SOTA provenance systems for both DBMS. Overall, TraceProv outperforms all other systems. The system scales well in both data size (we present results for TPC-H up to SF 100) and degree of available parallelism (up to 10 threads).

2 RELATED WORK

Several models for fine-grained provenance for relational queries have been proposed in past work [13]. *Lineage* [6] determines for each result tuple $t \in Q(D)$ of a query Q a subset of the input database D that is sufficient for producing t. For some use cases, we are just interested in a *witness* [4, 10, 18] which is a subset $D' \subseteq D$ of the input database that is sufficient: $Q(D') = Q(D)$. As computing a witness and, thus, also lineage, of minimal size is computationally hard [18], TraceProv, like other systems, does not guarantee that the lineage or witnesses it produces are minimal. Important use cases of provenance including probabilistic query evaluation [36], what-if queries [8, 24] and view maintenance [30], and how-to queries [5, 19, 21] require knowledge about how input tuples were combined to generate a result. The semiring annotation framework [17] models such information as annotations on data. Using the semiring of provenance polynomials, tuples are annotated with symbolic expressions that encode how input tuples (represented as variables) have been combined (addition represents disjunctive use while multiplication represents conjunctive use). For example, if a query result tuple is annotated with a polynomial $x^2 + x \cdot y$ then this encodes that there are two alternative ways to derive this result: (i) by joining the input tuple x with itself or (ii) joining x with y. The model has been extended beyond positive relational algebra [2, 7, 11, 12].

Systems for provenance capture on relational databases differ in the provenance models they support, how they represent provenance information internally, how provenance is captured, and what kind of queries over captured provenance are supported. Trio [1, 25] was one of the first systems for capturing lineage and was implemented as a python-based frontend to PostgreSQL that instruments SQL queries to call a UDF that internally executes queries to capture provenance. Perm [14–16] and GProM [3] compile SQL with provenance requests into plain SQL. GProM supports multiple database backends. Both systems use a flat representation of provenance polynomials for positive relational algebra and lineage for queries involving other language constructs such as aggregation and nested subqueries: each output row maps one result tuple to tuples in its provenance. DataProv [31] rewrites queries to produce provenance polynomials encoded as strings. ProvSQL [34] is a postgres extension that captures semiring provenance. Queries are instrumented by adding calls to UDFs that communicate with a background worker to write provenance polynomials as factorized arithmetic circuits mmaped files.

Smoke [32] is a native approach that captures lineage by piggybacking on existing data structures generated during query execution to log mappings between input and output tuples of individual operators in a query. As in our approach, these logs for individual operators have to be joined in a second phase to reconstructing lineage top-down. SmokedDuck [22, 23] is a similar approach implemented in DuckDB. Müller [9, 26, 27] captures lineage in two phases on PostgreSQL. The system uses UDFs to write logs to database tables. The system ensures that provenance is only logged once at the cost of having to run a query to check existence for every log record. The second phase of the system works bottom-up, which can be detrimental for queries that produce intermediate result with large provenance that is filtered out eventually.

3 OVERVIEW

TraceProv currently supports PostgreSQL and DuckDB. The system is bolt-on and leverages a database-independent logging framework. Thus, porting the approach to new databases is low effort. For instance, adding support for an older version of DuckDB for experiments against SmokedDuck required only 10 hours of work, pending optimizations. We support queries composed of arbitrarily SPJA queries with window functions, nested subqueries, and set operations. TraceProv computes the provenance of a query in two phases. The logging is executed if a user requests the system to capture provenance for a query Q. The system instruments Q propagate and write provenance to logs through calls to UDFs and UDAs that wrap the system's logging framework. With a few exceptions, TraceProv preserves the structure of the input query. Thus, unlike systems like GProM [3] which generate significantly more complicated capture queries that are hard to optimize, the instrumented query generated by TraceProv typically has the same execution plan as the input query. During rewriting, we also construct a plan which encodes the schema of the logs generated for the query, and which provenance attributes they share. The instrumented query is then executed using the database backend, causing the provenance of Q to be written to the logs stored in memory-mapped files or memory regions not backed by a file. We allow multiple threads / processes to concurrently write into the same logical log without synchronization via thread/process specific physical logs.

After logging has finished for a query Q, the user issue provenance requests for Q. TraceProv supports four types of requests: (i) return a *witness*, i.e., a subset of the tuples from the input dataset that is sufficient for producing the same result as Q; (ii) the second type of request returns the lineage of each individual query result tuple as a binary table mapping between output tuple ids and input tuple ids. (iii) the third type of request returns the lineage of a single result tuple; (iv) the last type of provenance request computes the provenance polynomial for each query result. For all types of provenance requests, TraceProv constructs one query per input relation R_i of the query that traces output tuple(s) to input tuples from R_i top-down by joining logs generated in the logging phase. Note that a separate query is constructed for each table access, even if the same table is accessed multiple times. For provenance polynomials a single query is constructed that returns the polynomials.

Backtrace queries are executed in DuckDB using table-valued UDFs for reading from logs. The plan generated in the first phase guides the system what logs were created and how they have to be joined. TraceProv can partition logs and automatically builds zonemaps for logs that are utilized by backtrace queries. TraceProv also supports parallel reading of logs which allows backtrace queries to leverage parallel query execution in DuckDB.

4 CAPTURE INSTRUMENTATION

In this section, we discuss how TraceProv instruments a query to log provenance information in the logging phase. The instrumentation uses a set of rewrite rules to inject propagation of provenance data and logging of propagated provenance into the input query. We present these rewrites in relational algebra.

4.1 Relational Algebra

We use relational algebra with bag semantics. We use common notation for operators: projection (Π), selection (σ), join ($\bowtie$), left outer join ($⟕$), set union ($\cup$), bag union ($\uplus$), set intersection ($\cap$), and bag intersection ($\intercal$). We allow projection to project onto a list of scalar expressions and apply renaming, e.g., $\Pi_{a+b\rightarrow c}(R)$ returns a single attribute c whose values are computed by adding attributes a and b. We denote aggregation grouping on attributes G and applying aggregation function $\mathcal{F}$ to input attribute a as $\gamma_{G;\mathcal{F}(a)}$. For aggregation without group-by we drop G. We also allow multiple aggregation functions to be used in a single aggregation. The duplicate elimination operator δ is denoted by δ. To model windowed aggregation supported in SQL, we introduce a window operator $\omega^{[l,u]}_{\mathcal{F}(a);G;O}(R)$ which extends each input tuple t from R with an additional attribute storing the result of computing aggregation function $\mathcal{F}$ on attribute a for a window (subset of R) determined as follows. We partition R on the partition-by attributes G and select the partition containing tuples t' where $t'.G = t.G$. Tuples within each partition are then ordered on attributes O, breaking ties arbitrarily. Finally, all tuples within $[l, u]$ of the position of t in the sort order are included in the window for t. For instance, `SELECT sum(a) OVER (PARTITION BY b ORDER BY c ROWS BETWEEN 3 PRECEDING AND CURRENT ROW) FROM R` can be expressed as $\omega^{[-3,0]}_{\textbf{\textit{sum}}(a);b;c}(R)$.

We allow nested subqueries inside selection and projection expressions. As in SQL, subqueries can reference attributes from the schema of the operator they are nested within. For a given input tuple t from an operator whose parameters contain a nested subquery, we substitute correlated attribute references within the nested subquery with values from t and then evaluate the subquery. We support the following types of nesting: scalar subqueries Q which evaluate to a single value, `EXISTS` subqueries $\Phi_{\exists}(Q)$ which evaluate to true if Q returns a non-empty result, and `ANY (ALL)` subqueries $\Phi[\exists t : \vec{a}\ \text{op}\ t](Q)$ ($\Phi[\forall t : \vec{a}\ \text{op}\ t](Q)$) which evaluate to true if comparison a op t evaluates to true for at least one (all) tuples t returned by Q. As an example, consider the SQL query shown below and the corresponding query in relational algebra.

```
SELECT a
FROM R
WHERE EXISTS (SELECT *
              FROM s
              WHERE R.b = S.c)
      OR a < 5;
```

$\Pi_a(\sigma_{\Phi_\exists(Q_1)\vee a<5}(R))$
$Q_1 = \sigma_{b=c}(S)$

4.2 Capture Plans and Log Functions

As mentioned previously, TraceProv combines propagation with logging of provenance attributes of a query instead of always propagating (like GProM) or always logging (like SmokedDuck). When instrumenting a query Q for provenance capture, TraceProv constructs a *plan* PLAN_Q that records at which operators of the query a log $\mathbf{l}$ will be written and what provenance attributes will be logged. Formally, a plan PLAN_Q is a set of logs $\{\mathbf{l}_i\}$ where each log is a tuple

$$\mathbf{l}_i = (\mathcal{E}, \mathcal{P}, p)$$

where i is a unique identifier for the log within the scope of the query. $\mathcal{P}$ are the provenance attributes of the log, i.e., the schema of the records that are written to the log. We use $\mathcal{T}$, $\mathcal{T}_1$, ..., to denote provenance attributes. $\mathcal{E}$ is a list of entries which each stores metadata about a single or group of provenance attributes that are logged at $\mathbf{l}_i$. Each entry in $\mathcal{E}$ further records the origin of these attributes (e.g., an input relation or another log), and their type (e.g., produced by an aggregation). As an example, consider the plan for our running example shown in Figure 2. $\mathbf{l}_1$ contains entries $\mathcal{T}_{\text{ID}(Q)}$, R($\mathcal{T}_{rid}$), and R($\mathcal{T}_{sid}$). The first entry ($\mathcal{T}_{\text{ID}(Q)}$)'s type encodes that the logged attribute is a tuple identifier for intermediate results produced by an operator (in this case, $\mathcal{L}_{\text{agg}}$ at the instrumented aggregation operator in the query). The latter two entries, R($\mathcal{T}_{rid}$), and R($\mathcal{T}_{sid}$), are base relation entries (type R) which store particular the renamed names of input relation attributes that are used as provenance attributes ($R_1.rid$ and $R_2.sid$ in the example). For $\mathbf{l}_1$, the schema of the log has one provenance attribute per entry, but in general one entry may correspond to multiple attributes, e.g., if a table has a multi-attribute key that are treated as provenance attributes. $\mathbf{l}_2$ contains only 1 entry, A($\mathcal{T}_{\text{ID}(Q)}$) of type A. This type signifies that the entry's provenance attribute was produced by a prior $\mathcal{L}_{\text{agg}}$ call which used attribute $\mathcal{T}_{\text{ID}(Q)}$ to store output tuple identifiers. Note that in visual depictions of plans, we connect a reference to an entry and the source of the entry with an edge, e.g., the edge between $\mathbf{l}_2$.A($\mathcal{T}_{\text{ID}(Q)}$) and $\mathbf{l}_1.\mathcal{T}_{\text{ID}(Q)}$. In the backtrace (see section 6 these edges are used to determine how to join logs. The plan serves two important roles: (i) it records entrys that encode the schemas of logs; (ii) it stores how the logs are related through entries.

4.2.1 Logging functions. The instrumented queries used by TraceProv for logging use two functions to append records to a log that we have already seen in the introductory example: $\mathcal{L}_{\text{slog}}(\mathbf{l}_i, c_1, \ldots, c_n)$ and $\mathcal{L}_{\text{agg}}(\mathbf{l}_i, c_1, \ldots, c_n)$. Here, $\mathbf{l}_i$ is the log to insert into and $c_1, \ldots, c_n$ are the provenance attribute value to be written to the log which are either propagated provenance attributes of input relations or, tuple identifiers for intermediate results. $\mathcal{L}_{\text{slog}}$ writes a log record and always returns true. Function $\mathcal{L}_{\text{agg}}$ is a UDA that returns a unique tuple identifier for a group in an aggregation and logs the

mapping between provenance attributes of aggregation's input and the generated tuple identifier.

4.3 Capture Rewrite Rules

TraceProv instruments queries for provenance capture using a set of rewrite rules. For a query Q we use Q^+ to denote the rewritten query after instrumentation using the rules. The most important rules are shown in Figure 3. The remaining rules are shown in [20]. In addition to returning Q^+, each rule computes the provenance attributes $\mathcal{P}$ propagated for the rewritten query, its log entries $\mathcal{E}$, and, optionally, creates a new log $\mathbf{l}$ in the plan for the query. The final plan PLAN_Q is then simply the set of generated logs. Each operator in a query is associated with a unique identifier with $\text{ID}(Q)$ denoting the identifier for the root operator of an algebra expression Q. These identifiers are used, e.g., to create unique names for provenance attributes. We use $\mathbf{R}$ to denote the schema of relation R and use the same notation for the schema of queries (e.g., $\mathbf{Q}$ is the schema of Q).

Table Access. Consider the rule **R1** for a table access operator. This operator is rewritten by adding a projection that propagates the primary key attributes PK(R) of relation R as provenance attributes using renaming to create unique names for each attribute as follows: an attribute $a \in \text{PK(R)}$ is renamed as $\mathcal{T}_{\text{ID}(R),a}$. Note that in the rewrite rule $\overrightarrow{\mathcal{T}_{\text{ID}(R)}}$ denotes the list of these new attribute names, and $||$ denotes concatenation of lists and, abusing notation, concatenation of individual elements to lists. No log is written at a table access operator.

Selection. A selection (**R4**) is rewritten by applying the selection to its rewritten input. However, selection conditions may contain nested subqueries which are rewritten by function rewrExpr which we will cover in Section 4.4. The provenance attributes (log entries) of a selection are the same as the provenance attributes (log entries). of its input.

Projection. For projection (**R5**), we append the provenance attributes of the input to the projection expressions. Recall that projection expressions are allowed to contain nested subqueries. Thus, like for selection, the expressions (and the contained subqueries) have to be rewritten.

Join. Joins are rewritten (**R2**) by joining the rewritten inputs of the join. The provenance attributes for the join is the concatenation of the provenance attributes of both inputs. The same applies to the log entries. No log is created for joins.

Bag union. For bag union (**R6**), TraceProv computes the union of the rewritten inputs. However, as the two rewritten inputs may differ in their number of provenance attributes, we pad the schema of the rewritten input with fewer provenance attributes with null values. This is done by using $\mathcal{NE}(Q_i, Q_j)$ which returns list of $\max(0, |(\mathbf{Q})_\mathbf{i}| - |\mathbf{Q_j}|)$ null values used for padding. Provenance attributes of Q_1 and Q_2 are unioned together which makes it ambiguous to determine which input provenance query an output tuple came from during backtrace. For disambiguation, we add new attribute that stores the identifier of the operator. The log entry for bag union records the name of the attribute identifying the input a tuple stems from as well as the lists of entries for both inputs of union. This information is used during backtrace to understand which attributes are derived from which inputs of the union. Further, $\mathcal{P}^{\max}_{Q_1,Q_2}$ returns the provenance attributes of either Q_1 or Q_2 whichever one is longer. No log is created for bag union.

Aggregation. For aggregation (**R3**), an output tuple for a group depends on all input tuples from that group. As mentioned earlier, TraceProv creates new tuple identifiers for aggregation outputs and stores the mapping between these new tuple identifiers and provenance attributes of input tuples to the aggregation. In the rule, we use $\mathcal{F}$ to denote a list of aggregation functions. The aggregation is rewritten by adding a call to log function $\mathcal{L}_{\text{agg}}$ which writes these mapping records to the newly created aggregation log ($\mathbf{l}_{\text{ID}(Q)}$). A new name $\mathcal{T}_{\text{ID}(Q)}$ is used for the attribute storing the tuple identifiers generated by $\mathcal{L}_{\text{agg}}$. The log entry for aggregation $\text{A}(\mathbf{l}, \mathcal{T}_{\text{ID}(Q)})$ stores the log $\mathbf{l}_{\text{ID}(Q)}$ created for the aggregation and the name of the tuple identifier attribute.

Left Outer Join. Left Outer Joins (**R7**) are rewritten similarly to joins, i.e., by joining the rewritten inputs of the left outer join. Similar to inner joins, the provenance attributes of the left outer join is the concatenation of the provenance attributes of both inputs. The same follows for the log entries. No log is created for joins.

Duplicate Removal. Duplicate removal (**R8**) is treated as a group-by aggregation without any aggregate functions. Thus, the rule decomposes to **R3**. The mapping records are written to the newly created aggregation log ($\mathbf{l}_{\text{ID}(Q)}$), and a new name $\mathcal{T}_{\text{ID}(Q)}$ is used for the attribute storing the tuple identifiers generated by $\mathcal{L}_{\text{agg}}$. The log entry for aggregation $\text{A}(\mathbf{l}, \mathcal{T}_{\text{ID}(Q)})$ stores the log $\mathbf{l}_{\text{ID}(Q)}$ created for the aggregation and the name of the tuple identifier attribute.

Set Union. Set union (**R9**) is treated as a bag-union (**R6**) followed by the duplicate removal (**R8**). The provenance attributes, log entries, and logged attributes are hence recursively defined. Log is created for set union, through application of **R8**.

Set Intersection. Set intersection (**R10**) is rewritten as an inner join (**R2**) followed by duplicate removal (**R8**). The resulting rewritten query performs a group-by aggregation over $\mathbf{Q}_1$. The inner join obtains the records that exist in both Q_1 and Q_2, while the duplicate removal obtains the unique instances of those records. Here, the concatenation of $\mathcal{P}(Q_1)$ and $\mathcal{P}(Q_2)$ is logged in the newly created aggregate log $\mathbf{l}_{\text{ID}(Q)}$. The propagated entry here is $\mathcal{T}_{\text{ID}(Q)}$ (output of $\mathcal{L}_{\text{agg}}$). The log entry $\text{A}(\mathbf{l}, \mathcal{T}_{\text{ID}(Q)})$ stores the log $\mathbf{l}_{\text{ID}(Q)}$ created for the aggregation and the name of the tuple identifier attribute.

Bag Intersection. Bag Intersection (**R11**), like **R10** for set intersection, obtains records that exist in both Q_1 and Q_2 through an inner join. The bag intersection needs to only remove at most m instances of a record from Q_1 where m is the number of times the record appears in Q_2. Thus, before the inner join, for each input query (Q_1 and Q_2) to the intersection, we give each occurrence of a record a unique ID (i.e., output of window function row number # where the window definition is partitioned on schema of the input query). The unique ID is annotated as $\mathcal{T}^1_\#$ and $\mathcal{T}^2_\#$ for Q_1 and Q_2 respectively, and the join condition is augmented to include $\mathcal{T}^1_\# = \mathcal{T}^2_\#$. This results in the correct number of removals of a record from Q_1. Here, the aggregate log function $\mathcal{L}_{\text{agg}}$ is used as window function which maps the input records to the tuple identifiers, and this mapping is stored in two different logs $\mathbf{l}_{\text{ID}(Q_1)}$ and $\mathbf{l}_{\text{ID}(Q_2)}$, and the generated tuple identifiers are referred to as $\mathcal{T}_{\text{ID}(Q_1)}$ and $\mathcal{T}_{\text{ID}(Q_2)}$.

Since two logs are generated, we insert into log nodes into the plan, $\mathcal{T}_{\mathrm{ID}(Q_1)} \,||\, \mathcal{E}(Q_1)$ and $\mathcal{T}_{\mathrm{ID}(Q_2)} \,||\, \mathcal{E}(Q_2)$.

Window Function. Window functions (**R12**) are rewritten by first storing the provenance attributes of the input rows ordered by the combination of window function's partition-by and order-by expressions. This is done by introducing a row number window function that orders by $G \,||\, O$, i.e., combination of window function's partition-by and order-by expressions. The output of row number is indicated by $\mathcal{T}^1_{\#}$. The provenance attributes are stored in this order by using the $\mathcal{L}_{\text{agg}}$ as a window function, with no partition-by expression, and $\mathcal{T}^1_{\#}$ as the order-by expression. The $\mathcal{L}_{\text{agg}}$ creates the new aggregate log $\mathbf{l}_{\mathrm{ID}(Q)}$, and outputs $\mathcal{T}_{\mathrm{ID}(Q)}$.

The $\mathcal{T}^1_{\#}$ is added to the order-by clauses of the original window frame definition and we use this extended frame definition to evaluate the original window function. We further add, to the extended frame definition, two new window functions $\mathrm{fv}(\mathcal{T}^1_{\#})$, and $\mathrm{lv}(\mathcal{T}^1_{\#})$. These functions evaluate the row number expression at the first row and last row of a window frame respectively. In this case, these functions return the first row number and the last row number that the window function frame uses to evaluate the original function. Thus, effectively encoding the intervals that belong in the provenance for each frame (and subsequently output of the window function).

The output of $\mathrm{fv}(\mathcal{T}^1_{\#})$ and $\mathrm{lv}(\mathcal{T}^1_{\#})$ is treated as a provenance attribute, and is propagated alongside $\mathcal{T}_{\mathrm{ID}(Q)}$. The log entry for window function $\mathrm{W}(\mathbf{l}, \mathcal{T}^{\mathrm{ID}(Q)}_{\mathrm{fv}}, \mathcal{T}^{\mathrm{ID}(Q)}_{\mathrm{lv}})$ stores the $\mathbf{l}_{\mathrm{ID}(Q)}$ created for the window function and the name of frame start and frame end identifier attributes.

The final instrumented query Q_{log} used by TraceProv to capture provenance for Q may require an additional final logging step unless the rewrite rules create a log for the root operator of the query below. We use $\mathbf{l}(Q)$ to denote the log, if any, written by the root operator of query Q. Q_{log} is defined as shown below.

$$Q_{log} = \begin{cases} Q^+ & \textbf{if } \mathbf{l}(Q^+) \neq \varnothing \\ \Pi_{\mathcal{Q}^+, \mathcal{L}_{\text{slog}}(\mathbf{l}_{\mathrm{ID}(Q)}, \mathcal{P}(Q))}(Q^+) & \textbf{otherwise} \end{cases}$$

Example 2 (Query instrumentation). *Figure 4 shows the relational algebra tree for the running example query, the query instrumented for provenance capture, and plan. The two table access operators were rewritten by adding a projection to duplicate and rename their primary key attributes. These attributes are written to log $\mathbf{l}_1$ at the rewritten aggregation using log function $\mathcal{L}_{agg}$. Then finally as the root of the rewritten query (selection) does not have an associated log, another projection is added in Q_{log} to log which aggregation results did fulfill the selection condition.*

Example 3 (Query with Nested Subqueries instrumentation). *Figure 5 shows the algebra tree for a nested subquery. The original query is shown at the bottom while the rewritten query is shown at the top. There are three query blocks ①, ②, and ③ corresponding to the levels of nesting: 0-level (i.e, top-level), 1-level, and 2-level respectively. Similar to Example 2, the three table access operators are rewritten by adding a projection to duplicate and rename their primary key attributes (in this case, rid, sid and tid).*

*The rewriter first gets called on the query block ①, and it discovers the subquery ② through Rule **R4**, and rewrite on ② gets triggered. Subsequently, the subquery ③ gets discovered, through Rule **R4** again. Further, for ③, we also apply the rewrite rule **R3**.*

In the $\mathcal{L}_{slog}$ for each query block, we include the provenance attributes of the outer query blocks. For example, for query block ③, we include $\mathcal{T}_{rid}$ and $\mathcal{T}_{sid}$ from 1 and 2 in $\mathcal{L}_{slog}$. Similarly, for query block ②, we include $\mathcal{T}_{rid}$ from 1 in $\mathcal{L}_{slog}$, which creates a new correlation between query blocks 1 and 2 which was absent in the original query.

Here, $\mathbf{l}_1$ is generated via $\mathcal{L}_{agg}$ and the logs $\mathbf{l}_2$, $\mathbf{l}_3$ and $\mathbf{l}_4$ are generated via $\mathcal{L}_{slog}$. Further, the plan not only encodes implicit relationships created via $\mathcal{L}_{agg}$, such as relationship between $\mathbf{l}_1$ and $\mathbf{l}_2$, but also explicit relationships created via introduced or existing correlations such as relationship between $\mathbf{l}_4$ and $\mathbf{l}_3$, and between $\mathbf{l}_3$ and $\mathbf{l}_2$.

4.4 Rewriting Expressions and Subqueries

Recall that we allow condition selections and projection expressions to contain nested subqueries which may be correlated to the operator in which the subquery occurs. The subquery will be evaluated for each tuple from the outer operator. When logging provenance for a tuple t_{inner} produced by one such evaluation of the nested subquery, we need to record for which tuple t_{outer} from the outer query this tuple was produced such that during backtrace the provenance of t_{outer} can be traced back to the provenance of t_{inner}. TraceProv achieves this by adding additional correlations with provenance attributes from the outer query that are included in the top level log written inside the nested subquery.

Let $\Phi(Q)$ denote a subquery of one of the supported types, e.g., $\Phi_{\exists}(Q)$ or $\Phi[\exists t : a \ \mathsf{op}\ t](Q)$. For a scalar expression e, we use subqueries(e) to denote the subqueries that occur in e and $e[e_1 \leftarrow e_2]$ to denote replacing all occurrences of e_1 in expression e with e_2. Recall that within the scope of a query we identify each operator with a unique identifier. Thus, substituting a nested subquery in an expression replaces only this specific instance of the nested subquery even if the same subquery expression appears more than once in the expression. Algorithm 2 shows the pseudocode for function rewrExpr$(e, \mathcal{P})$ called from the algebraic rewrite rules to rewrite projection expressions and selection conditions. The function takes as input an expression e to rewrite and the provenance attributes $\mathcal{P}$ from the outer query. It replaced every subquery $\Phi(Q)$ in e with a rewritten version which is generated by rewriting Q and then uses $\mathcal{L}_{\text{slog}}$to write the the provenance attributes from the outer query and the provenance attributes of Q to a new log.

It is sometimes possible for TraceProv to determine that an expression evaluates to a fixed result irrespective of the result of a nested subquery. For instance, in $\sigma_{a=3 \vee \Phi_{\exists}(Q)}(R)$ for any tuple t from R with $t.a = 3$ the selection condition evaluates to true independent of whether $\Phi_{\exists}(Q)$ returns true or false. If we can detect such situations efficiently, then we can avoid logging provenance for this specific execution of the nested subquery to reduce the size of the captured lineage / witness. Consider a selection condition θ containing a nested subquery $\Phi(Q)$. To apply this optimization, TraceProv creates a *guard* θ_{guard}. If the guard evaluates to false then θ's result is independent of the result of the nested subquery and, thus, the provenance for this execution of the nested subquery does

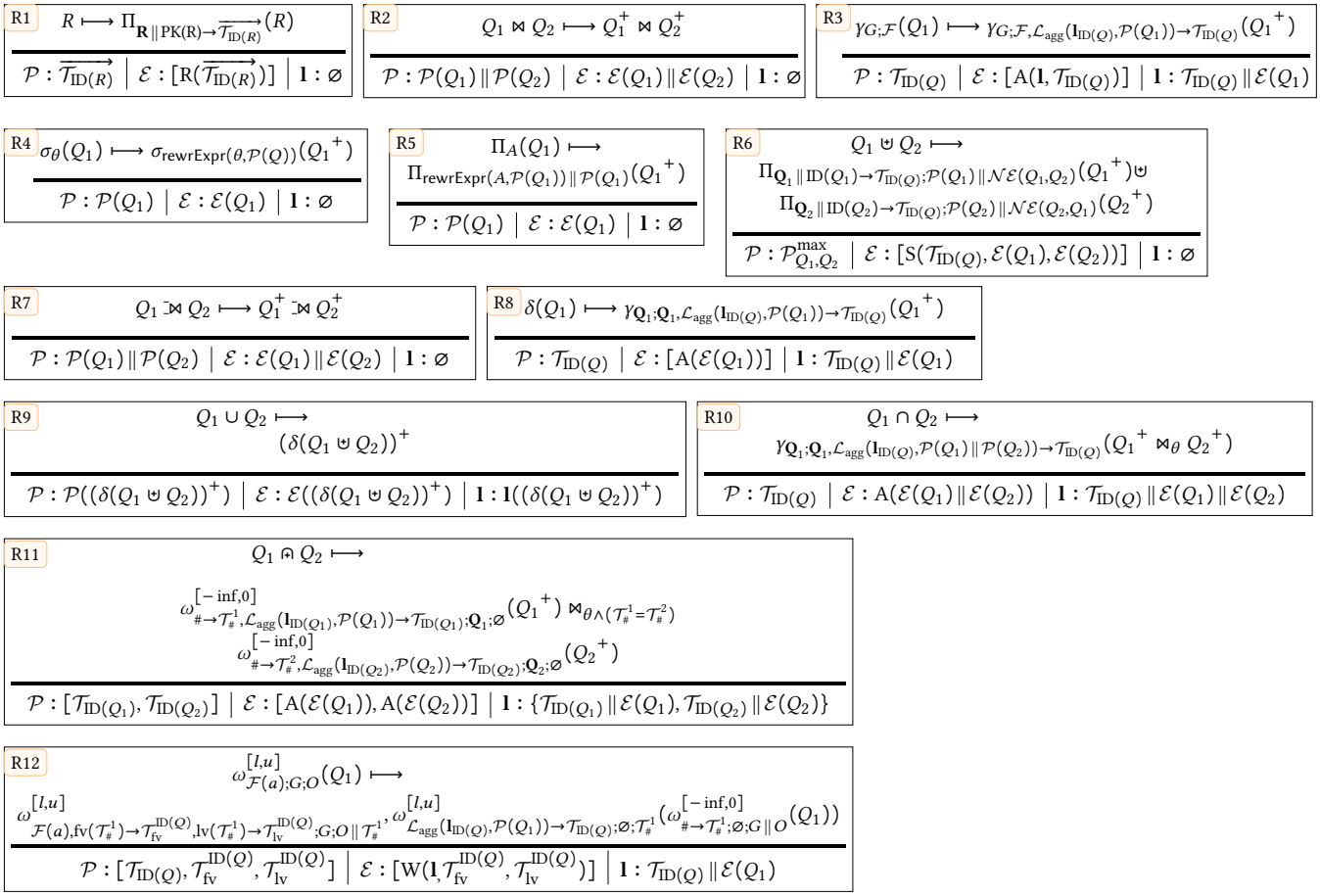


Figure 3: Rewrite rules for instrumenting queries for provenance capture

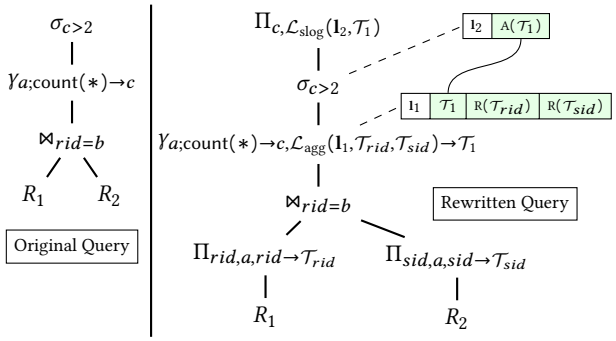


Figure 4: Running example plan and instrumented query.

not have to be captured. The guard is then injected into the rewritten nested subquery as follows to avoid writing to the subquery's top-level log if the guard fails:

$$\Pi_{\mathbf{Q}}(\Pi_{\mathcal{Q} \| \mathcal{L}_{\text{slog}}(\mathbf{l}_{\text{ID}(Q)},\mathcal{P} \| \mathcal{P}(Q))}(\sigma_{\theta_{\text{guard}}}(Q^+)))$$

We now explain how the θ_{guard} is constructed. Suppose θ is the boolean expression used in *R4*. The algorithm for generating θ_{guard} is shown in Algorithm 1, for a given subquery $\Phi(Q)$. We first convert θ into its DNF form θ_{DNF}, where $\theta_{\text{DNF}} = \vee_i \theta_i = \vee_i \wedge_j \theta_{i,j}$. Here, θ_i refers to a clause while $\theta_{i,j}$ denotes a literal.

Algorithm 1 generateConPredicate($\Phi(Q), \theta$)

$\theta_{\text{DNF}} \leftarrow \vee_i \wedge_j \theta_{i,j}$ *// get DNF form of θ*
$\theta_{\text{guard}} \leftarrow \theta$
if $\Phi(Q)$ in $\theta_{m,n}$ **then**
  $\theta_{\text{guard}} \leftarrow \wedge_{i \neq m}(\neg\theta_i) \wedge (\wedge_{i=m,j\neq n}(\neg\theta_{i,j}))$
return θ_{guard}

For some types of subqueries, we add more and clauses to the θ_{guard} obtained from Algorithm 1. For example, for $\Phi[\exists t : a \text{ op } t](Q)$ and $\Phi[\forall t : a \text{ op } t](Q)$, we add the test expression to θ_{guard}, i.e., $\theta_{\text{guard}} \leftarrow \theta_{\text{guard}} \wedge (a \text{ op } \Pi_{\mathbf{q}}(q))$

5 LOGGING FRAMEWORK

The performance of TraceProv significantly impacts the system's performance. Thus, it is critical that the cost of writing a log record is minimized and logging scales in the degree of available parallelism. We now discuss the several design decisions made to achieve this goal.

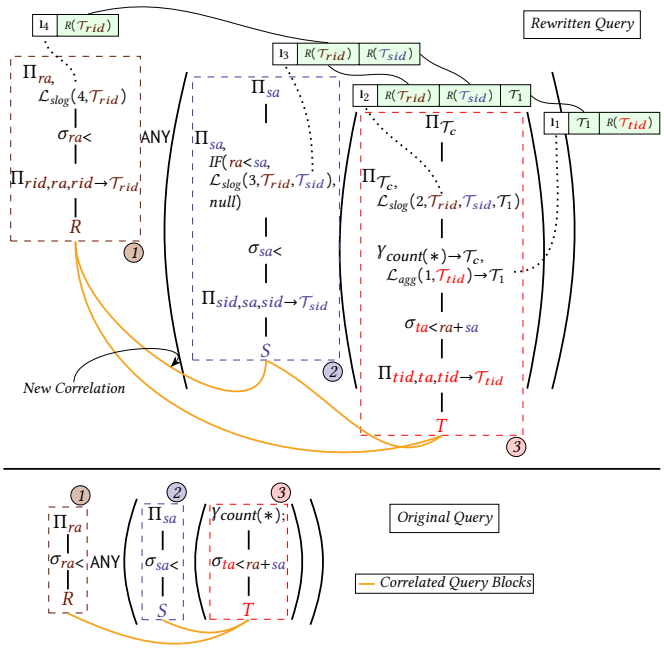


Figure 5: Nested Subquery Example

Algorithm 2 rewrExpr$(e, \mathcal{P})$

for $\Phi(Q) \in$ subqueries(e) **do**
  $\Phi(Q)^+ \leftarrow \Pi_{\mathbf{Q}}(\Pi_{\mathcal{Q} \parallel \mathcal{L}_{\text{slog}}(\mathbf{l}_{\text{ID}(Q)}, \mathcal{P} \parallel \mathcal{P}(Q))}(Q^+))$
  $e \leftarrow e[\Phi(Q) \leftarrow \Phi(Q)^+]$
return e

5.1 The Log Interface

Recall that TraceProv uses UDFs and UDAs to append log records to a log, and that the locations of these logs are decided by the rewriter. The logging framework exposes two vectorized functions:

- **UDF** $\mathcal{L}_{\text{slog}}(\mathbf{l}_i, c_1\text{: int[}], c_2\text{: int[}] \ldots c_n\text{: int[]}) \rightarrow$ True
- **UDA** $\mathcal{L}_{\text{agg}}(\mathbf{l}_i, c_1\text{: int[}], c_2\text{: int[}] \ldots c_n\text{: int[]}) \rightarrow$ int64

Here, $\mathbf{l}_i$ is a log, and $c_1, c_2 \ldots c_n$ are the attribute values of the log records to be appended to $\mathbf{l}_i$. Each c_i is vector of int(int32 or int64). The above representation allows unified handling of DuckDB and PostgreSQL. For DuckDB, UDFs and UDAs are called one chunk at a time. For PostgreSQL, the row-based execution can be modelled by forcing $|c_i| = 1$. UDA $\mathcal{L}_{\text{agg}}$'s purpose is to associate provenance attributes for tuples belonging group in the input of an aggregation to an identifier for the output tuple generated for this group. $\mathcal{L}_{\text{agg}}$ simply generates a unique group identifier (int64) for a group. As we want to avoid communication between parallel workers for this, each worker maintains a separate counter and we include the id for a worker and the log in the group identifiers to ensure they are unique within the scope of a query. UDA $\mathcal{L}_{\text{agg}}$extends each input log records with this group tuple identifier to store the association between the output tuple and contribution input tuples of the aggregation in the log it is appending to.

Recall that for a given $\mathbf{l}_i$, $\mathcal{L}_{\text{agg}}$can be executed concurrently by different workers. Since the counter is worker-local, no synchronization is needed to assign group identifiers locally. The group identifiers are made globally unique, without synchronization, by setting the 1 byte in the 8-byte representation of group identifier as the worker id. Note that worker id identifies a given thread or process in its corresponding thread/process pool. It is possible that the same group occurred across workers. In such cases, databases call a combiner to combine partial states for the same group from different workers. We leverage this mechanism to associate which group identifiers across workers are associated to the same group. Note that combine is treated as a second level aggregation, i.e., calling $\mathcal{L}_{\text{agg}}$ again with two attributes (source, and target), thus does not extend the log representation.

5.2 Physical Log Structure

We partition each *logical log* $\mathbf{l}_i$ into multiple *physical logs*, one for each worker. We use $\mathbf{pl}_{i,w}$ to denote the physical version of logical log $\mathbf{l}_i$ for worker w. Every worker thread (or process in the case of PostgreSQL) involved in processing a query appends to its private physical log without having to communicate or synchronize with other workers. The only exception is a single round of synchronization at the start of a query to assign each worker an id from a global counter.

We now discuss the physical representation of the log, which needs to represent chunks of records stored in columnar form $c_1, c_2, \ldots, c_n$. Figure 6 shows the physical structure of a log which consists of 3 mmaped files (or mmaped memory regions): (i) the *data file* stores for each chunk of log records, the concatenation of column vectors $c_1, c_2 \ldots c_n$; (ii) the *stats file* stores statistics (number of records, minimum and maximum values for each column) for each chunk in the data file; (iii) the a *metadata file* storing global information about the physical log such as minimum and maximum column values across all chunks, the number of chunks, and pointers to the end of the data and stats files which are used when appending log records.

Note that core of logging is a simple memcpy operation from c_i to the current data file position. Hence, the cost per log record is in the order of nanoseconds (see Section 7.9). The storage size per log record for c_i is either 8 bytes or 4 bytes, dictated by the maximum number of bytes needed for representation of c_i. We support huge pages (2MB) to avoid expensive page-faults.

TraceProv can further partition a worker's physical log based off any column $(c_1, c_2 \ldots c_n)$. Currently, we only utilize partitioning for aggregation, partitioning on the group identifiers created by $\mathcal{L}_{\text{agg}}$. Figure 8 an example of appending a chunk to a partitioned physical log: log records from the chunk processed by a worker w that belong to a particular bucket are appended as a chunk to the physical log for this bucket.

As explained in Figure 3, each *logical* log has a unique identifier. A *logical* log may consist of multiple *physical* log that again have an identifier that is unique within the scope of a query. The rationale for dividing a logical log into multiple physical logs is to enable multiple workers (e.g., threads in DuckDB or processes in PostgreSQL) to write to the same logical log without synchronization and to partition the log for partition pruning during backtrace. The

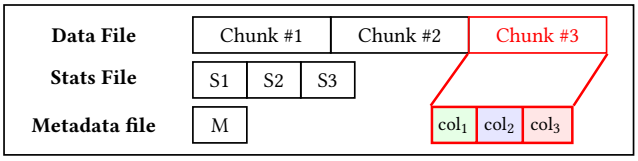


Figure 6: Physical log structure: (i) the data file stores columnar chunks of log records; (ii) the stats file records statistics for each chunk such as min/max values per column; (iii) the metadata file stores global metadata for the physical log.

earlier is critical for the system to scale in the degree of available parallelism. TraceProv uses 8 bytes integers as identifiers for input, intermediate result, and output tuples. Figure 6 shows the structure of a physical log for a columnar database such as DuckDB, consisting of three mmaped files: (i) the data file stores the actual log records split into chunks. Within each chunk the records are stored in a columnar fashion; (ii) the stats file stores statistics (number of records, minimum and maximum values for each column) for each chunk in the data file; (iii) a metadata file storing global information about the physical log such as minimum and maximum column values across all chunks, the number of chunks, and pointers to the end of the data and stats files which are used when appending records to the log.

Log records. A log record consists of one or more columns which can each store 8 byte values. For example, ignoring space required for the statistics and metadata, a log logging 100 pairs of input tuple identifiers, e.g., each recording two input tuples that were joined, would have 16-byte records and have a total size of $16 \cdot 100 = 1,600$ bytes. For row-based databases, there is no subdivision of the data file into chunks and the data file uses a row-based layout. As log records are of a fixed size for a given log, the row-based storage layout does not incur any additional storage overhead over the columnar layout.

Appending to a Log. In a columnar database, log records are written in a batched fashion, i.e., one chunk at a time. Appending a batch of records essentially just requires: (i) copying arrays of fixed size tuple identifiers for each column of the log to create a new chunk at the end of the data file, (ii) advancing the data pointer (end of the data file) to point to the end of the new chunk; (iii) computing the statistics for the chunk and appending them to the end of the statistics file; (iv) advancing the pointer for the statistics file; and (iv) updating the log metadata. The data and statistics files for a physical log are grown whenever needed by a constant number of pages (32 per default) at a time. For row-based database systems, log records are appended one record at a time.

5.3 Logging Function Implementations

The logging infrastructure of TraceProv is exposed to the database as two UDFs $\mathcal{L}_{\text{slog}}$and $\mathcal{L}_{\text{slog}}$as well as a UDA $\mathcal{L}_{\text{agg}}$.

Scalar logging. For scalar logging, $\mathcal{L}_{\text{slog}}(\mathbf{l}_i, c_1, \ldots, c_n)$ is a variadic function that takes as input the id of a logical log and the attribute values c_1 to c_n of a log record. Independent of its input this function always returns true. For columnar databases like DuckDB that support vectorized UDFs we implement $\mathcal{L}_{\text{slog}}$ as such a function. Algorithm 3 shows the pseudocode for the vectorized implementation of $\mathcal{L}_{\text{slog}}$ that takes a vector of log records Rand the logical log $\mathbf{l}_i$ to write too. This function may be called for this logical log in parallel by multiple workers of the database that each will write to a separate physical log. For that we first determine from which worker this function is called. Each worker maintains a local data structure (a thread-local data structure for DuckDB and a global variable for PostgreSQL where workers correspond to processes) that stores its id. Function getWorkerID retrieves this id or, if it has not been determined yet, fetches the next available id from a global counter. This counter is the only data structure for which we have to prevent concurrent access from multiple workers. However, every worker only accesses this counter once during the execution of a query. Given the workers id w, we can then determine the physical log $\mathbf{pl}_{w,i}$ that w should write to. We then appends the log records R to that log.

Algorithm 3 $\mathcal{L}_{\text{slog}}(\mathbf{l}_i, R)$

```
1 w ← getWorkerID()
2 append(pl_{w,i}, R)
```

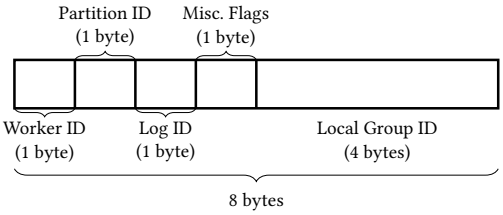


Figure 7: Aggregate state for $\mathcal{L}_{\text{agg}}$ encoding a globally id for a group that is unique within the scope of a query.

Aggregate Logging. For aggregations, we further partition the physical log based on a hash of the group-by attribute into a configurable number of fragments. This then enables partition pruning during backtrace when joining a parent log of the aggregation log with the aggregation log/ The log UDA $\mathcal{L}_{\text{agg}}$ used for aggregate logging serves a dual purpose: it writes log records for every input tuple belonging to a group (or every input tuple for aggregation without group-by) and it generates identifiers for each output tuple of the aggregation (its return value). In SQL, a UDA consists of a state, an initialization function, an update function that updates the state based on the values from an input record, and a finalize function that computes the final result of the aggregation function based on the current state. In $\mathcal{L}_{\text{agg}}$, the state is a 64-bit unsigned integer that is used to store an output tuple identifier. As shown Figure 7, such a tuple identifier consists of multiple parts including the id of the worker creating the output, the partition the group belongs to, the logical log id, several flags, and a worker + partition local group id. The purpose of splitting the output tuple id into multiple components is to enable unique identifier to be generated for output tuples without requiring coordination between workers. The initialization function of $\mathcal{L}_{\text{agg}}$sets that state to an invalid output tuple identifier (0).

Algorithm 4 Vectorized update function $\mathcal{L}_{\text{agg}}^{update}(\mathbf{l}_i, R, states)$

```
w ← getWorkerID()
for i ∈ {1, ..., |R|} do
    t ← R[i]
    state ← states[i]
    g ← getGroupID(state)
    if g = 0 then                    // Create new worker-local group ID?
        g ← incrGrpNum(w, i)
        p ← h(g)
        flags ← 0
        state ← constructGroupID(w, p, i, flags, g)
    g ← getGroupID(state)
    p ← h(g)
    append(pl_{w,i,p}, g, t)
```

Aggregate update function. Algorithm 4 shows the vectorized version (DuckDB) of the update function for UDA $\mathcal{L}_{\text{agg}}$. This function gets as input a batch of log records R to process and a batch of aggregation function states $states$ containing for each log record $t \in R$ the state for the group that t belongs too. After retrieving the workers id, the function iterates over each $t \in R$ to update the state for the group that t belongs too. We first extract the 4-byte group id g from the state for t. If this is the first update call for this group, then $g = 0$. In this case, a new worker-local group id is generated by incrementing a worker-local counter for log $\mathbf{l}_i$. We then determine the partition p that g belongs to using hash function h and store in $state$ a new group identifier that uniquely identifies this group within $\mathbf{l}_i$ as it contains the worker id, partition, log id i and worker-local group id g. Finally, we append to the physical log for worker w and partition p, a log record storing the association between an output group id and the input provenance attributes from t.

Aggregate finalizer function. The finalizer function for $\mathcal{L}_{\text{agg}}$ does simply return the global group number stored in the state for this group to be used as the tuple identifier for this group in the output of the aggregation.

Example 4 (Update function). *Figure 8 shows an example of processing a batch of updates. For each input record in the batch, we assign a group id β_i if necessary and determine the physical log it will be written to. All log records in the batch that belong to this partition are appended as a new chunk to the physical log's data file, the statistics for these log records are appended to the log's statistics file and the metadata for the physical log is updated.*

Aggregate combiner function. Most database systems that support parallel aggregation, split aggregation into two phases: the input is divided among workers that create partial aggregation results for groups based on their input and then these partial results are combined to produce the final aggregation result for each group. In TraceProv, if a group is processed by more than one worker during pre-aggregation, it will be assigned different group ids by each of these workers. The combiner function for $\mathcal{L}_{\text{agg}}$assigns a new global id for each group that will serve as the tuple identifier for this group in the output of the aggregation (and will be returned by $\mathcal{L}_{\text{agg}}$'s finalizer function). Furthermore, it writes to a separate physical combiner log which worker-local group ids correspond to a final group id. Algorithm 5 shows the pseudocode for the combiner function. The input is a batch of local aggregation states R produced by a workers during pre-aggregation and the aggregation states for the combined output $states$. For each local state in the batch, if no global id for this group has been assigned we use the group id from the local state. In any case the association between local and global group id is written to a physical combiner log $\mathbf{pl}_{w,i}^{comb}$ specific to the worker w running the combiner function and logical log $\mathbf{l}_i$.

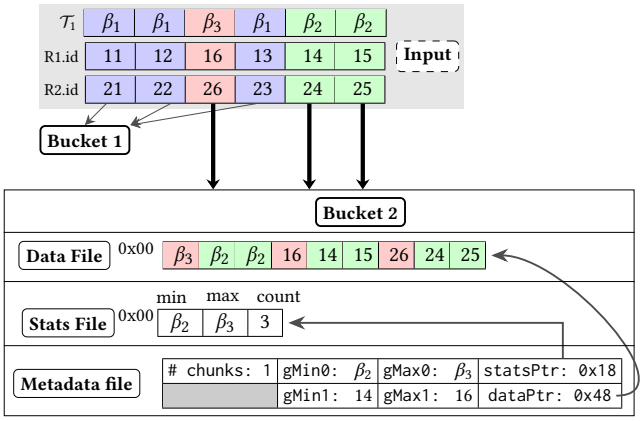


Figure 8: Structure of a partitioned physical log for aggregation. The log records belonging to Bucket 2 have been logged as a chunk to this bucket's physical log.

Algorithm 5 Vectorized combiner function $\mathcal{L}_{\text{agg}}^{combine}(\mathbf{l}_i, states, R)$

```
w ← getWorkerID()
for i ∈ {1, ..., |R|} do
    state_global ← states[i]                  // global group id
    state_worker ← R[i]                       // worker-local group id
    g_global ← getGroupID(state)
    if g_global = 0 then                      // No global id assigned yet?
        state_global ← state_worker           // reuse local group id
    append(pl^comb_{w,i}, state_global, state_worker)
```

6 BACKTRACE

Backtrace is the second phase of TraceProv which returns the lineage of the input query Q using one or more queries $\mathbb{Q}_{BT}$. Each such query traces back from the result of the input query Q to one of the input relations accessed by Q by joining logs generated in the first phase. Recall that during the first phase we construct a plan PLAN_Q which records which logs are written during logging, their schemas, and additional metadata. Thus, PLAN_Q determines how the logs have to be joined to trace query output tuples back to contributing tuples from a particular input relation. The queries in $\mathbb{Q}_{BT}$ are always executed using DuckDB. For PostgreSQL, this is done within a table access method.

Algorithm 6 backtrace(PLAN_Q)

```
1 𝕋_BT ← ∅
2 l_root ← ROOT(PLAN_Q)                    // Get top-level log
3 Q_root ← ℛ(l_root)                       // Read top-level log
4 backtraceEntries(Q_root, l_root, 𝕋_BT)
5 return 𝕋_BT
```

Algorithm 7 backtraceEntries ($Q_{cur}, \mathbf{l}_{cur}, \mathbb{Q}_{BT}$)

```
1  for l_sub ∈ subQ(l_cur) do
2      backtraceSub(Q_cur, l_sub, 𝕋_BT)
3  for L ∈ l_cur do
4      if L = R(T_1, ..., T_n) then
5          𝒬_BT[R] ← Π_{ℐD(Q), T_1, ..., T_n}(Q_cur)
6      else if L = A(l_child) then
7          backtraceAgg(Q_cur, l_child, 𝕋_BT)
8      else if L = W(l_child) then
9          backtraceWin(L, Q_cur, 𝕋_BT)
10     else if L = S(s_id, [l_1, ..., l_k]) then
11         backtraceSet(L, Q_cur, 𝕋_BT)
```

6.1 The Log Reader

Recall that logs are stored in a custom binary format in mmaped areas in memory, optionally backed by files. To access logs from within a backtrace query $\mathbb{Q}_{BT}$, we provide a table-valued function $\mathcal{R}$ (the *log reader*) that returns a log as a duckdb table. The log reader is implemented as a table function $\mathcal{R}(\mathbf{l}_i, \text{offset})$ that takes as input a log $\mathbf{l}_i$ it should read from and optionally an offset. The optional offset is used when computing the lineage for a single result tuple (see Section 6.2). DuckDB allows parallel reads from table functions. We currently assign one DuckDB thread each physical partition of the logical log $\mathbf{l}_i$. We use lightweight statistics generated during logging for partition pruning and data skipping in backtrace. If parameter offset is provided, then only the row at this position will be returned. Note that in TraceProv the top-level log always contains a single row per result tuple. Thus, retrieving the entry for the ith result tuple corresponds to reading the ith row in the log.

6.2 Backtrace Query Construction

We will first describe the backtrace query construction when computing the lineage for every query result tuple as binary tables and then discuss how other request types are handled. The main entry point is backtrace which takes a plan PLAN_Q for a query Q as input and returns $\mathbb{Q}_{BT}$ which associates each table access R in Q with a query $\mathbb{Q}_{BT}[R]$. If a relation R is accessed more than once, e.g., in a self-join $R \bowtie R$, a separate query will be created for each access. backtrace initializes an empty map for $\mathbb{Q}_{BT}$ and creates an initial query Q_{cur} that reads the top-level log, i.e., the root of PLAN_Q. It then calls backtraceEntries which processes the entries of a log.

If a log has attributes from an operator that contains a nested subquery, then the log for this subquery is connected the outer log. backtraceEntries first iterates over all such logs connected to $\mathbf{l}_{cur}$ (denoted as $\text{subQ}(\mathbf{l}_{cur})$) and uses function backtraceSub to process the logs for these nested subqueries. Afterwards, the algorithm processes all entries of $\mathbf{l}_{cur}$, dispatching to methods for handling specialized entries ($\mathbf{l}(\mathcal{T})$ returns the log to which provenance attribute $\mathcal{T}$ belongs to). For an entry L that corresponds an input relation, i.e., is of the form $\text{R}(\mathcal{T}_1, \ldots, \mathcal{T}_n)$, backtrace, we project the current query on the output tuple identifier attribute for the query ($\mathcal{T}_{\text{ID}(Q)}$) and the relation's provenance attributes $\mathcal{T}_1, \ldots, \mathcal{T}_n$ of L.

Algorithm 8 backtraceAgg($Q_{cur}, \mathbf{l}_{cur}, \mathbb{Q}_{BT}$)

```
1 Q_cur ← Q_cur ⋈ ℛ(l_cur)                 // join with aggregate log
2 if iscombined (l_cur) then               // parallel agg. with combiner?
3     l_preagg ← getPreaggLog(l_cur)
4     Q_cur ← Q_cur ⋈ ℛ(l_preagg)          // join with combiner log
5     l_cur ← l_preagg
6 backtraceEntries(Q_cur, l_cur, 𝕋_BT)
```

Aggregate Logs. Function backtraceAgg (Algorithm 8) implements backtrace for aggregation. It first joins the aggregate log $\mathbf{l}_{cur}$ with the current backtracing query Q_{cur}. If the database used parallel aggregation where multiple workers pre-aggregate the data locally followed by a combiner step, then TraceProv generates two logical logs, both of which are physically partitioned by worker. In the parallelized aggregation case, $\mathbf{l}_{cur}$ in line 1 of Algorithm 8 refers to $\mathbf{l}_{comb}$. We then need to join with the log $\mathbf{l}_{preagg}$ for the pre-aggregation step (lines 3-6). Afterwards, backtraceEntries is used to process the entries of the pre-aggregation or single log for the aggregation.

Example 5 (Backtrace Query Construction). *Reconsider our running example query. Figure 4 shows the algebra tree for this query and the two logs generated during logging. $\mathbf{l}_2$ records which aggregation results pass the filter and, thus, are in the final query result while $\mathbf{l}_1$ does map groups (each group is assigned an identifier stored in $\mathcal{T}_{ID(Q)}$) to pairs of tuples from R_1 and R_2 belonging to this group in the aggregation's input. backtrace will generate two queries, each tracing back from the query result to one of the input relations by joining the log top-down. These queries are constructed by first initializing Q_{cur} with a log reader for $\mathbf{l}_2$ (the top-level log) in the main backtrace algorithm backtrace. Then backtraceEntries is called with this initial query. Function backtraceEntries then iterates over all entries of log $\mathbf{l}_2$. There is a single entry of type A (an aggregation result log). This entry is processed by backtraceAgg by joining the current query (reading log $\mathbf{l}_2$) with the log referenced in the entry ($\mathbf{l}_1$), updating Q_{cur} to $\mathcal{R}(\mathbf{l}_2) \bowtie \mathcal{R}(\mathbf{l}_1)$. Then backtraceEntries is called again to handle the entries of $\mathbf{l}_1$. When processing the entry for $R(\mathcal{T}_{rid})$, the backtrace query for R_1 is created. Similarly, when processing $R(\mathcal{T}_{sid})$ we create the query for R_2, resulting in the final state of $\mathbb{Q}_{BT}$. All the backtrace queries are shown in Equation* (1)

$$\begin{aligned} \mathcal{Q}_{BT}[R_1] &= \Pi_{\mathcal{T}_1, \mathcal{T}_{rid}}(\mathcal{R}(\mathbf{l}_2) \bowtie \mathcal{R}(\mathbf{l}_1)) \\ \mathcal{Q}_{BT}[R_2] &= \Pi_{\mathcal{T}_1, \mathcal{T}_{sid}}(\mathcal{R}(\mathbf{l}_2) \bowtie \mathcal{R}(\mathbf{l}_1)) \end{aligned} \tag{1}$$

Window Function Logs. Recall that for window functions, TraceProv creates a log that for each window records the start and end position of input tuples of the window function in the child log. During backtrace the log entries between these positions have to be received for each window. For that TraceProv implements a

table function $\mathcal{R}_{win}(\mathbf{l}, i, j)$ which reads log entries between i and j from $\mathbf{l}$. Note that as rows in a log are of fixed size. Thus, the physical location of the ith can be determined upfront without having to scan the log. Algorithm 9 shows the pseudocode of function backtraceWin which handles window entries. This function uses a dependent join to join Q_{cur} using the startand endattributes from Q_{cur} to fetch all entries from the log belonging to the window. It then processes the entries of this log using backtraceEntries.

Algorithm 9 backtraceWin($\mathrm{W}(\mathbf{l}_{child}, start, end), Q_{cur}, \mathbb{Q}_{BT}$)

1 $Q_{cur} \leftarrow Q_{cur} \bowtie \mathcal{R}_{win}(\mathbf{l}_{child}, start, end)$
2 backtraceEntries($Q_{cur}, \mathbf{l}_{child}, \mathbb{Q}_{BT}$)

Union. Recall that for unions we instrument each input to propagate an indicator as one of the provenance attributes that enables us to determine which input of the union a tuple originates from. These inputs of the union may differ in their number of provenance attributes. In the logging query we make the inputs union-compatible by padding the inputs with null values. backtraceSet shown in Algorithm 10 handles a union entry $\mathrm{S}(s_{id}, [\mathbf{l}_1, \ldots, \mathbf{l}_k])$. To trace back to a specific input of the union, we have to filter on the indicator attribute s_{id} to ensure that only outputs derived from this input are tracked. Then the filtered query is traced to the corresponding using backtraceEntries using the entries $\mathbf{l}_i$ for this branch of the union.

Algorithm 10 backtraceSet($\mathrm{S}(s_{id}, [\mathbf{l}_1, \ldots, \mathbf{l}_k]), Q_{cur}, \mathbb{Q}_{BT}$)

for $i \in [1, k]$ **do**
  $Q_{filter} \leftarrow \sigma_{s_{id}=i}(Q_{cur})$
  backtraceEntries($Q_{filter}, \mathbf{l}_i, \mathbb{Q}_{BT}$)

Algorithm 11 backtraceSub($Q_{cur}, \mathbf{l}_{sub}, \mathbb{Q}_{BT}$)

$Q_{cur} \leftarrow Q_{cur} \bowtie \mathcal{R}(\mathbf{l}_{sub})$
backtraceEntries($Q_{cur}, \mathbf{l}_{sub}, \mathbb{Q}_{BT}$)

Handling Nested Subqueries. Nested subqueries are handled by backtraceSub (Algorithm 11) that is passed as input the top-level log $\mathbf{l}_{sub}$ for the subquery. This log is then natural joined with the current query Q_{cur}. Note that common attributes between Q_{cur} and $\mathbf{l}_{sub}$ correspond to correlations between the nested subquery and the outer query traced by Q_{cur}. Joining on these attributes is necessary to match the provenance of outer tuples with tuples from the corresponding evaluation of the nested subquery. Afterwards, the entries of $\mathbf{l}_{sub}$ are processed using backtraceEntries to continue tracing further. In [20] we explain how set operations and window functions are handled during backtrace.

Other Request Types. To handle requests for a witness for a query Q, we adjust line 5 in Algorithm 7 to exclude the output tuple identifier attribute ($\mathcal{T}_{\mathrm{ID}(Q)}$) from the projection omitting the information of which output an input tuple contributes. For a request for the lineage of a single result tuple t, we adjust line 3 in Algorithm 6 to add the offset for t to the call of the log reader to restrict the output to the provenance of t. As in this case the log reader will return a single row, we can execute this log read and then replace the join with the next log (if any) with a selection. We refer to this optimization as the *join rewrite* as a join is rewritten into a selection. For provenance polynomial requests a single query is constructed that joins all logs, injecting UDFs to construct a single polynomial for each result row.

Example 6 (Backtrace for nested subqueries). *We now explain how backtrace for the subquery Example 3 is performed. Here, backtrace queries are generated for R, S, and T.*

We first initialize Q_{cur} with log reader for $\mathbf{l}_4$ (the top-level log in the backtrace) in the main backtrace algorithm backtrace. Then backtraceEntries is called with this initial query ($\mathcal{R}(\mathbf{l}_4)$), and the $\mathbf{l}_4$. (Algorithm 7). On line 1 of Algorithm 7, we iterate over all the logs from subqueries that are connected to $\mathbf{l}_4$, i.e., $\mathbf{l}_3$. Now, we call backtraceSub with the current query Q_{cur}, and $\mathbf{l}_3$. backtraceSub creates a natural join between the current Q_{cur} (which is $\mathcal{R}(\mathbf{l}_4)$) and $\mathcal{R}(\mathbf{l}_3)$. Note that the common attributes here are $\mathcal{T}_{rid}$, which is used in the join equality condition, i.e., $\mathcal{R}(\mathbf{l}_4).\mathcal{T}_{rid} = \mathcal{R}(\mathbf{l}_3).\mathcal{T}_{rid}$. Now, backtraceEntries is called with this updated join query.

Now, log subqueries connected to $\mathbf{l}_3$ (i.e., $\mathbf{l}_2$) are iterated over, and backtraceSub is called again. The log reader $\mathcal{R}(\mathbf{l}_2)$ is initialized, and the common attributes between $\mathbf{l}_2$ and Q_{cur} are $\mathcal{T}_{rid}$ and $\mathcal{T}_{sid}$. These common attributes are used in the subsequent join operation between Q_{cur} and $\mathbf{l}_2$. backtraceEntries is called with the updated join query, and $\mathbf{l}_2$.

There are no subquery logs connected to $\mathbf{l}_2$ and we iterate over the entries. We find two entries of type R, which we ignore since they are introduced as part of correlation. We then find entry of A, which is processed by backtraceAgg by joining the current query ($\mathcal{R}(\mathbf{l}_4) \bowtie \mathcal{R}(\mathbf{l}_3) \bowtie \mathcal{R}(\mathbf{l}_2)$) with the log referenced in the entry ($\mathbf{l}_1$), updating the Q_{cur} to $\mathcal{R}(\mathbf{l}_4) \bowtie \mathcal{R}(\mathbf{l}_3) \bowtie \mathcal{R}(\mathbf{l}_2) \bowtie \mathcal{R}(\mathbf{l}_1)$. Then, backtraceEntries is called to handle the entries of $\mathbf{l}_1$. When processing the entry for $R(\mathcal{T}_{tid})$, the backtrace query for T is created.

Recall that so far the backtraceEntries has only processed subqueries for logs $\mathbf{l}_3$ and $\mathbf{l}_4$. Now, backtraceEntries will process the entries for these logs which include $R(\mathcal{T}_{sid})$ and $R(\mathcal{T}_{rid})$ respectively. This results in the backtrace queries being generated for S and R respectively, in that order. All the backtrace queries are shown in Equation (2).

$$\begin{aligned} \mathbb{Q}_{BT}[R] &= \Pi_{\mathcal{T}_{rid}}(\mathcal{R}(\mathbf{l}_4)) \\ \mathbb{Q}_{BT}[S] &= \Pi_{\mathcal{T}_{sid}}(\mathcal{R}(\mathbf{l}_4) \bowtie \mathcal{R}(\mathbf{l}_3)) \\ \mathbb{Q}_{BT}[T] &= \Pi_{\mathcal{T}_{tid}}(\mathcal{R}(\mathbf{l}_4) \bowtie \mathcal{R}(\mathbf{l}_3) \bowtie \mathcal{R}(\mathbf{l}_2) \bowtie \mathcal{R}(\mathbf{l}_1)) \end{aligned} \tag{2}$$

7 EXPERIMENTAL EVALUATION

In this section, we evaluate TraceProv against current state-of-the-art system to evaluate its provenance capture performance for a variety of workloads on PostgreSQL and DuckDB, comparing against SOTA systems on both databases.

7.1 Experimental Setup & Workloads

All experiments are run on Mac Studio machine with a M2 Max 12-core CPU and 64GB of RAM. Each experiment is performed

15 times. We report median runtimes, excluding the first 5 executions (hot cache). We use PostgreSQL version 16, and DuckDB (version `1.4.3`). We mainly focus on **lineage capture** as lineage is defined for a large class of relational operations including nested subqueries, aggregation, and window functions. We consider two variants: `P_all` computing lineage for all result tuples of a query and `P_single` which only requires capture for one result tuple (selected randomly unless stated otherwise). To demonstrate the broad applicability of our techniques, we also consider **provenance polynomials**: a more expressive type of provenance that encodes how input tuples were combined to derive a query result and are needed for use cases like probabilistic query processing [34, 36] and hypothetical reasoning [8, 21, 24]. TraceProv supports provenance polynomials for SPJU queries.

7.1.1 Baselines. **PostgreSQL.** For PostgreSQL, we compare against the rewrite-based system *GProM* [3], the approach proposed in [26] which we refer to as *Müller*, and *ProvSQL* [33, 34]. As mentioned in section 2, Müller [26] utilizes a two-phase approach that computes provenance bottom-up in the second phase while TraceProv backtracks starting from the query result (top-down). ProvSQL stores provenance polynomials as circuits which are stored in memory-mapped files. ProvSQL uses a separate background worker process to log provenance to these mmaped files. For parallel query execution, each worker process has to communicate with this background worker, limiting performance in parallel settings.

DuckDB. We also use **GProM** and the native lineage capture system SmokedDuck [22, 24]. **SmokedDuck** also splits lineage capture into two phases and performs the second phase top-down like TraceProv. However, in contrast to TraceProv which uses UDFs and a separate logging framework, SmokedDuck is deeply integrated into DuckDB's execution engine, being based on DuckDB version 0.10.3. For a fair comparison, in experiments directly comparing against SmokedDuck, we run TraceProv on the same DuckDB version. Currently, SmokedDuck does not support multi-threaded query execution. SmokedDuck exists in multiple versions. We contacted the authors and based on the guidance selected *smokedduck-2025-f*. However, the second phase of this branch either crashed or returned incorrect lineage for most of the TPC-H queries. We fixed some of these bugs to be able to compare against this system.

7.1.2 Metrics. We report the execution time as measured by the database system. For PostgreSQL, we the sum of "Execution Time" and "Planning Time" as reported by running the query with `EXPLAIN ANALYZE (timing off)`. For DuckDB, we report `latency` (or `timing` for SmokedDuck) after turning on query profiling. In some experiments, we also report storage size of the logs written by a system.

7.2 TPC-H Overview

TPC-H SF10. Figure 9 shows the runtime of lineage capture for TraceProv and all baselines on TPC-H SF10 for lineage capture for a single row (`P_single`) and for all result tuples (`P_all`). In the plots, we group queries by their structure. For each query we also report the runtime of the query without provenance capture as a reference point (NoProv). We report the total end-to-end time for each system (that means the sum of both phases for TraceProv, Müller, and SmokedDuck) and a timeout of 10 min on PostgreSQL and 5 min on DuckDB. On the right of each plot we show the distribution of the runtime of TraceProv relative to the fastest system per query. For GProM, we used window-based rewrites with heuristic optimizations enabled. This setting of GProM provides the best performance compared to other GProM settings for most TPC-H queries.

TraceProv outperforms all other systems, often by a large margin, on both PostgreSQL and DuckDB. When computing lineage for all result rows (`P_all`) on average TraceProv is 2.0x (2.4x) faster than the fastest system per query on DuckDB (PostgreSQL). When only the lineage of a single result row is requested, TraceProv is still on average 1.2x (2.0x) faster on DuckDB (PostgreSQL). Note that for some TPC-H queries, SmokedDuck does not return correct lineage as it does not include any tuples required for nested subqueries to evaluate to true in the lineage. This impacts the results of queries *Q2*,*Q4*,*Q17*,*Q18*,*Q20*,*Q21*,*Q22*. Even though SmokedDuck only returns a subset of the lineage for these queries, TraceProv is still more efficient. GProM is reasonably competitive for simpler queries with aggregation and few joins. In fact, is the only system to outperform TraceProv on some TPC-H queries: for DuckDB these are *Q14* (`P_all`) and *Q1*, *Q12*, *Q14*, *Q9* (`P_single`). These are queries for which GProM outperforms NoProv because it optimizes away the final aggregation step of the query, an optimization that we do not support yet in TraceProv. However, when considering all TPC-H queries, GProM is not competitive. The system exhibits larger overhead on queries with a larger number of joins and is not competitive for queries with nested subqueries due to its flat relational encoding of provenance. For example, TraceProv outperforms GProM by several OOM for queries with multiple nested subqueries, where GProM's flat encoding enumerates all combinations of input tuples from the provenance of each nested subquery.

TraceProv outperforms the native SmokedDuck on all queries with a factor 0.9x to 3.6x on `P_single`, the task for which SmokedDuck is more competitive. This is largely due to the cost SmokedDuck pays for indexing captured provenance between the first and second phase. As we will expand on further in Section 7.4, the cost of indexing is hard to amortize, even when lineage captured in phase one is used to serve multiple retrieval requests (executions of phase two). Note that the indexing cost was excluded in the reported measurements in [22, 23], leading to different conclusions. Müller is more competitive than GProM on queries with nested subqueries as it uses a factorized representation, but is also heavily outperformed by TraceProv (factor 0.8x to 7x on `P_all`) due to its high logging cost (writing a log record entails inserting a row into a database table with an additional test to check whether the row already exists) and the disadvantage of using a bottom-up approach in the second phase which generates lineage for all intermediate results of the query even if they do not end up contributing to the final query result. This is particular obvious for top-k queries such as Q3 and Q10. Finally, ProvSQL is not competitive which is due to the fact that it captures a more expressive provenance model (provenance polynomials), the large amount of data it writes to disk, and the design choice to use a single background worker process for logging polynomial circuits that all processes involved in a query have to communicate with. Note that the system does not support several TPC-H queries as it does not support top-k queries, nested

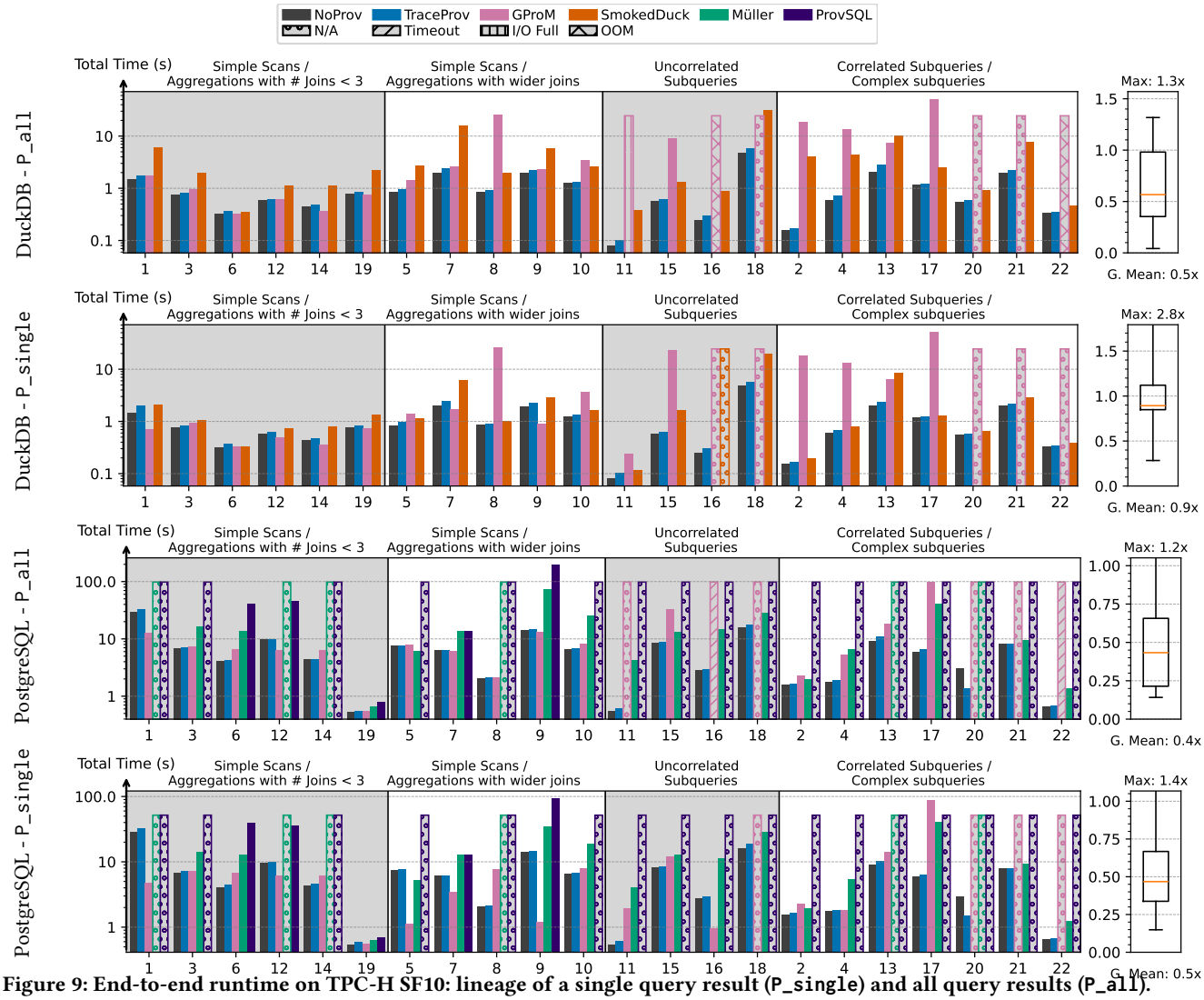
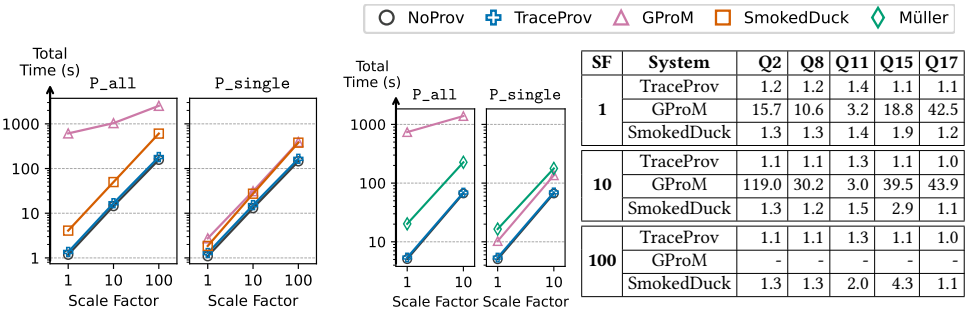


Figure 9: End-to-end runtime on TPC-H SF10: lineage of a single query result (`P_single`) and all query results (`P_all`).



(a) DuckDB **(b) PostgreSQL**

SF	System	Q2	Q8	Q11	Q15	Q17
1	TraceProv	1.2	1.2	1.4	1.1	1.1
	GProM	15.7	10.6	3.2	18.8	42.5
	SmokedDuck	1.3	1.3	1.4	1.9	1.2
10	TraceProv	1.1	1.1	1.3	1.1	1.0
	GProM	119.0	30.2	3.0	39.5	43.9
	SmokedDuck	1.3	1.2	1.5	2.9	1.1
100	TraceProv	1.1	1.1	1.3	1.1	1.0
	GProM	-	-	-	-	-
	SmokedDuck	1.3	1.3	2.0	4.3	1.1

(c) DuckDB `P_single` slowdown

SF	System	Q1	Q11	Q13	Q22
1	TraceProv	1.1	1.4	1.4	1.2
	GProM	0.4	2.4 K	3.4	1.9 K
10	TraceProv	1.1	1.1	1.2	1.0
	GProM	0.43	-	2.0	-

(d) PostgreSQL `P_all` slowdown

Figure 10: Cumulative end-to-end time for all 22 TPC-H queries varying dataset size (SF)

subqueries, and multi-level aggregation. For supported queries, TraceProv is faster 1.4x to 13.3x faster than ProvSQL.

Scalability in dataset size. We now evaluate how each system scales in dataset size. Figure 10a and Figure 10b show the cumulative runtime for all 22 TPC-H queries for SF=1, SF=10, and SF=100 for `P_all` and `P_single`. On all configurations, cumulative time of TraceProv is comparable to NoProv cumulative time. Indeed, across all the scale factors, tasks and database configurations, the total cumulative time of TraceProv is no more than 1.19x than the cumulative time of the baseline. Increasing the scale factor exposes

the limitations of competing systems. We present the slowdown of systems over NoProv for queries that GProM does not scale for in Figure 10c and Figure 10d for DuckDB `P_single` and PostgreSQL `P_all`, respectively.

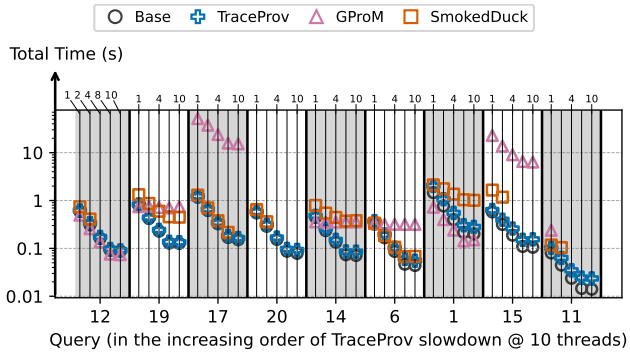


Figure 11: DuckDB TPC-H SF10 absolute query runtime varying the number of threads from 1 to 10 (shown on top).

7.3 Parallel Query Execution

TraceProv is designed to efficiently support parallel query engines, and we now evaluate its performance as parallelism is varied. We only consider DuckDB as it more effectively exploits available parallelism than PostgreSQL, and, thus, is more challenging. We compare against GProM and SmokedDuck on TPC-H SF=10. Recall that SmokedDuck's backtrace and index building does not utilize parallelism. In Figure 11 we vary the number of threads and compare against SmokedDuck and GProM for `P_single`. We only include those queries where SmokedDuck does not crash for at least one multithreaded setting. The queries are arranged in the order of increasing TraceProv slowdown (compared to the baseline) at 10 threads. TraceProv's per-query slowdown over NoProv for 10 threads ranges from 1.04x to 1.68x. Further, with increasing parallelism, TraceProv's performance over competing systems significantly increases. For example, for *Q19*, TraceProv is 10% (86ms) slower than GProM and is 60% (494ms) faster than SmokedDuck. However, for 10 threads, TraceProv is 449% (605ms) faster than GProM, and is 232% (313ms) faster than SmokedDuck. GProM scales poorly for queries *Q19*, *Q14*, *Q6*. For these queries, while GProM execution time remains almost constant, TraceProv continues to scale.

Figure 12 presents a comparison against GProM on DuckDB `1.4.3`for the `P_all`. Note that SmokedDuck is pinned to version `0.10.3`, and hence is excluded from these results. In Figure 12, we split the queries into three buckets based on TraceProv slowdown of that query, compared to baseline, under number of available threads set to 10. The first category has slowdown between 1.04x and 1.1x. The second category has slowdown between 1.1x and 1.2x. Finally, the third category has slowdown between 1.2x and 1.73x.

As evident, 7 TPC-H queries have the slowdown at 10 threads less than 10%, and a 14 queries have the slowdown less than 20%. Further, only 8 TPC-H queries have a higher slowdown between 1.2x and 1.73x, which is the maximum slowdown for TraceProv over base query execution. Observations from Figure 11 still hold. For example, TraceProv is 9.3% (72ms) slower than GProM for *Q19* for single-threaded execution. However, at 10 threads, TraceProv is 433% (518ms) faster than GProM.' Similarly, for *Q2*, TraceProv is 4x faster at 10 threads compared to single-threaded execution, while GProM is only 1.02x faster. This results in TraceProv being 630x faster than GProM for *Q2* at 10 threads. For queries such as *Q14* and *Q6*, TraceProv is worse than, or comparable to, GProM for single-threaded execution. However, at 10 threads, TraceProv is 3x and 5.5x faster than GProM for queries *Q14* and *Q6* respectively.

7.4 Amortization of SmokedDuck Indexing Cost

As SmokedDuck is the only native and also the most competitive in terms of overall performance, we present a more detailed comparison against this system. Recall that SmokedDuck is significantly slower than TraceProv for TPC-H SF10 even for `P_single` due to its limitation to a single thread and due to the large cost of indexing the captured lineage after logging. As the created index structures can improve the runtime of retrieving lineage in backtrace, a reasonable question to ask is whether it is possible for this overhead to be amortized across multiple requests, e.g., a user iteratively explores the lineage of individual query result tuples resulting in a sequence of backtrace requests. To answer this question, we first show the individual runtimes for logging and backtrace for both systems and then evaluate amortization across multiple requests.

Figure 13 we show the breakdown of runtime into logging and backtrace (`P_single`) for TPC-H SF10. Recall that SmokedDuck does not return full lineage for some TPC-H queries as it does not return any lineage for nested subqueries. For a fair comparison, we modified TraceProv to return the same lineage as SmokedDuck for this experiment. Setup includes indexing captured lineage after logging for SmokedDuck and SQL query generation and static determining what partitions to prune (for cases where we use partition pruning) for TraceProv. The queries are sorted by the relative capture overhead of TraceProv compared to NoProv. Even for single-threaded execution, TraceProv generally outperforms SmokedDuck if we measure the total end-to-end time. In terms of capture cost both systems exhibit similar performance (SmokedDuck did crash for Q16 during backtrace). This demonstrates the efficiency of TraceProv's logging framework that is competitive with SmokedDuck, even through SmokedDuck has direct access to DuckDB's execution engine internals. The setup cost for SmokedDuck is significantly higher than for TraceProv due to building fine-grained indexes over the logs captured in logging including hash tables for efficient backtracing in backtrace and TraceProv only has to pay a data-independent cost for generating the queries for both phases. While TraceProv builds lightweight data structures for backtrace (zone maps for data skipping and partitioning of for partition pruning) this is done during logging with low overhead. For backtrace, the differences between the systems are more pronounced. For some queries, mostly with smaller lineage, the fine-grained index structures constructed for SmokedDuck pay off while TraceProv benefits light-weight data skipping for heavier queries. Furthermore, due to its combination of propagation and logging, TraceProv benefits from having to join less logs during backtrace, e.g., SmokedDuck will create a log for each join of a query while TraceProv will create a single log.

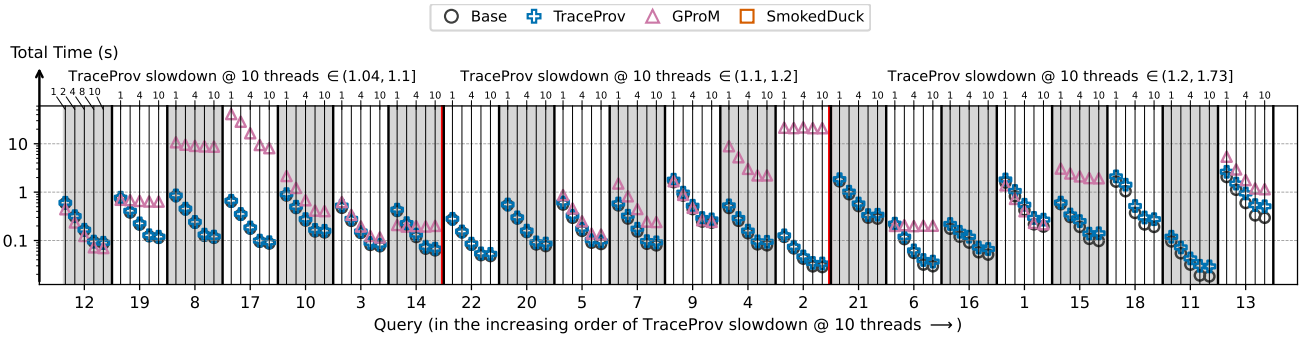


Figure 12: DuckDB varying the degree of parallelism. We report absolute runtime (top) and, for TraceProv, scalability relative to the scalability of the baseline system without lineage capture (bottom), i.e., y=1 implies the system scales as well as the vanilla DuckDB.

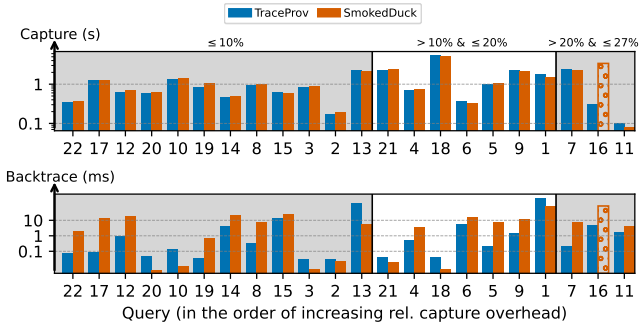


Figure 13: Logging and backtrace runtime of TraceProv and SmokedDuck for `P_single` on TPC-H SF10. Queries are ordered by relative capture overhead of TraceProv over NoProv.

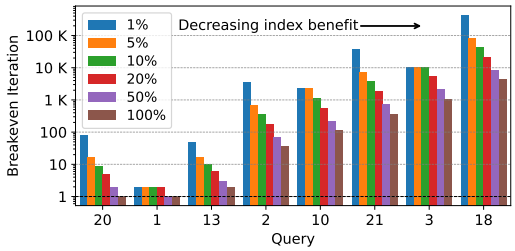
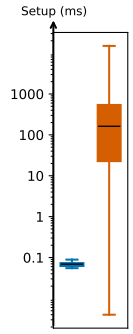
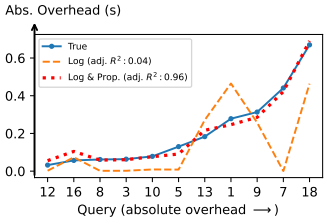


Figure 14: DuckDB - number of lineage requests (each requesting provenance for x% of query results) needed for SmokedDuck to break even with TraceProv.



(a) Setup cost distribution of TraceProv and SmokedDuck



(b) Analytical analysis of the capture cost of TraceProv. The blue solid line is the true absolute overhead and the red dotted line is the predicted overhead using a linear model over the amount of propagated and logged provenance data. It is evident that the predicted overhead aligns closely with the true overhead.. Queries are arranged in the order of increasing absolute overhead.

Figure 15

To evaluate amortization of indexing cost, we capture lineage for one of the TPC-H queries and then repeatedly request the lineage for a randomly selected fraction of the query's result (1% up to 100%). We report the mean number of such requests that have to be served before the two systems break even, repeating each experiment 1,000 times. Obviously, it is not possible for SmokedDuck to break even for queries for which TraceProv is faster in backtrace and, thus, these queries are excluded. The results are shown in Figure 14 where queries are sorted based on decreasing index benefits. For 14 out of 22 TPC-H queries, TraceProv is faster in backtrace than SmokedDuck. For the remaining 8 queries, it is technically possible for SmokedDuck to outperform TraceProv when the later is limited to single-threaded execution. However, the number of requests (requesting the lineage of each result row 10s to 1000s of times) that are necessary to amortize indexing are unlikely to occur in practice.

7.5 Detailed Analysis of SmokedDuck's index building cost

Why is SmokedDuck's index building slow?

Depending on the query plan and the physical operators used, SmokedDuck builds different types of indexes. We explain how SmokedDuck builds the index for the hash group-by. Since SmokedDuck tracks provenance by logging the data flow per chunk, the index for hash group-by tracks *which* chunks, and *which* rows *within* those chunks belong to a group. Specifically, it builds a dictionary $\mathcal{I}\{q \rightarrow \mathcal{C}\}$. $\mathcal{I}$maps the keys q(the group identifier) to $\mathcal{C}$, which is also a dictionary. $\mathcal{C}$\{ch_idx → row_id[]\} maps the chunk index (ch_idx) to the list of row ids (local to a chunk), the row_id[].

During lineage query (Backtrace phase), SmokedDuck probes the index $\mathcal{I}$with $q$group to determine the chunks that contain rows belonging to the group. Next, it derives the lineage further for each chunk. 12 presents the pseudocode of how the SmokedDuck's index builder for hash group-by. SmokedDuck's log is modeled as a 2-D list $\mathcal{L}$with type $\mathcal{G}$[chunk_idx][row_idx], where $\mathcal{G}$is the type used to represent group - `uint64_t` in SmokedDuck.

Algorithm 12 SmokedDuck hash group-by index

```
1 idx ← I
2 for each chunk_id ∈ |L| do
3     for each row_id ∈ |L[chunk_id]| do
4         G group_id : L[chunk_id][row_id]
5         idx[group_id][chunk_id].append(row_id)
```

For notational convenience, we denote $\sum_{i=0}^{|\mathcal{L}|}\sum_{j=0}^{|\mathcal{L}[i]|} 1$ as $||\mathcal{L}||$, i.e., total number of rows across all chunks. While the index structure is useful, SmokedDuck builds this index in an inefficient manner:

(1) SmokedDuck builds the index for all the groups, even if they got filtered later in the query evaluation. Hence, the dictionary $\mathcal{I}$contains keys that will never get queried in the Backtrace phase. This effects TPC-H queries such as *Q3* and *Q10* where not all the groups survive the top-N operation.
(2) While the theoretical complexity is $||\mathcal{L}||$, note that if the groups are spread randomly across the log $\mathcal{L}$, then on line 5 of pseudocode 12, a random vector will be appended to, for each iteration of inner loop (line 3). We show that this causes a significant performance penalty to SmokedDuck since random memory addresses will be written to in each access. In our microbenchmarks, we show that if the log is perfectly ordered with the group, then the index building time significantly drops. However, in general, this cannot be guaranteed.
(3) Note that the data access pattern also depends on the chunk size, i.e., the number of rows per chunk. DuckDB has can store upto 2048 rows per chunk (default value). However, operators such as filter, and join may make the chunk sparse, if only a few rows in a chunk pass the operator. DuckDB attempts to merge multiple chunks into one, but it only does so if the chunk has less than 64 rows (hardcoded limit). Given a constant source table cardinality (with fixed selectivity), sparse chunks implies that the number of chunks over all increases. While $||\mathcal{L}||$ (theoretical complexity) remains same regardless of chunk density (number of rows per chunk), sparser chunks increases absolute count of chunks, and thus a given group is likely distributed over multiple chunks, leading to more random memory accesses in line 5 of Algorithm 12.

TraceProv avoids unfruitful work, such as building indexes for artifacts that do not contribute to the provenance. This is done implicitly in the top-down approach. For example, the TraceProv equivalent to building the SmokedDuck index $\mathcal{I}$is the building the hash table on the group keys that *contributed* to the final result.

7.5.1 Factors that influence TraceProv capture performance. Recall that the rewriten query for TraceProv differs from the original query, at least for TPC-H, in only two possible ways: 1. introduction of logging functions, and 2. more propagagted attributes. Hence, we attempt to model the absolute overhead of TraceProv's capture phase over baseline across the TPC-H using the combination of these two features. For simplicity, we only consider queries for which the physical plan of TraceProv capture query does not differ from the original query, except for logging functions and extra propagagted attributes. We use the following linear regression model to estimate the absolute overhead in Equation (3). We use #col($\mathbf{l}_i$) to denote the number of columns of log $\mathbf{l}_i$, and $|\mathbf{l}_i|$ for the number of rows in a log. Further, we use #call($\mathbf{l}_i$) to denote the number of function calls by DuckDB for $\mathbf{l}_i$. We use $\overrightarrow{\text{Log}}$, and $\overrightarrow{\text{Join}}$ to represent set of all logs and all joins (in the physical plan) of a query. For each join $j \in \overrightarrow{\text{Join}}$, j.left and j.right refers to left and right side of the join.

$$T_{\text{ovh}} = \alpha_{\text{call}} \sum_{l\in\overrightarrow{\text{Log}}} \#\text{call}(\mathbf{l}_l) + \alpha_{\text{mater.}} \sum_{l\in\overrightarrow{\text{Log}}} |\mathbf{l}_l| \times \#\text{col}(\mathbf{l}_l)$$
$$\alpha_{\text{L_prop}} \sum_{j\in\overrightarrow{\text{Join}}} |\mathcal{P}(j.\text{left})| + \alpha_{\text{R_prop}} \sum_{j\in\overrightarrow{\text{Join}}} |\mathcal{P}(j.\text{right})| + \alpha_{\text{C}} \tag{3}$$

In Equation (3), we treat number of log calls as a separate feature from log size for simplicity. Recall that chunks in DuckDB may not be completely filled, and as chunks get sparse, more function calls may be necessary to log a constant number of rows, leading to more overhead from the function calls.

The second expression ($\alpha_{\text{mater.}}$), accounts for the memory copy (or logging) operations.

The third and fourth expression ($\alpha_{\text{L_prop}}$and $\alpha_{\text{R_prop}}$) account for the propagation cost of extra provenance attributes. We cost propagation from left and right side of a join differently since DuckDB uses irregular memory access when propagating columns from right side of a join. Finally, $\alpha_{\text{C}}$is an extra constant.

We fit the above model using Numpy's `linalg.lstsq` (least-squares solution to linear matrix equation) over TPC-H queries *Q1*, *Q3*, *Q5*, *Q7*, *Q8*, *Q9*, *Q10*, *Q12*, *Q13*, *Q18*. Figure 15b shows the true and predicted absolute overhead . The adjusted R^2 score of our model is 0.96. Note that if $\alpha_{\text{L_prop}}$and $\alpha_{\text{R_prop}}$are dropped from Equation (3), then the adjusted R^2 drops to 0.04, demonstrating the significance of including propagation costs in the estimation model. For example, consider *Q7* and *Q9*. The cumulative log sizes (accounting for # columns) is 408,559 and 22,831,466 respectively. Further, *Q9*'s total log call count is 14,692 which is significantly higher than *Q7*'s log count of 31. However, the true absolute overhead of *Q7* (440ms) is higher than that of *Q9* (313ms). This occurs because of the disjunction used in *Q7*, for which DuckDB propagated 18,230,325 rows before application of the disjunction predicate which outputs 58,365 rows that get logged during the aggregation. In this case, even though the log size is small, the cost of propagating new provenance attributes for 18.2M rows dominates the overhead.

7.5.2 Hash Aggregate. For completeness we also compare SmokedDuck on hash aggregation benchmark used in [23]. The benchmark consists of single query $Q_{\text{zipf}} = \gamma_{z;sum(v)}(T)$. The schema of T is

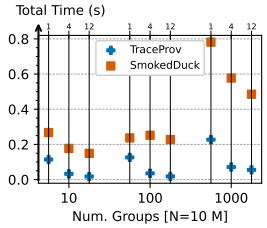


Figure 16: Average total end-to-end time to derive lineage of a row in the result of Q_{zipf}. We report the measurements for multiple threads 1, 4 and 12.

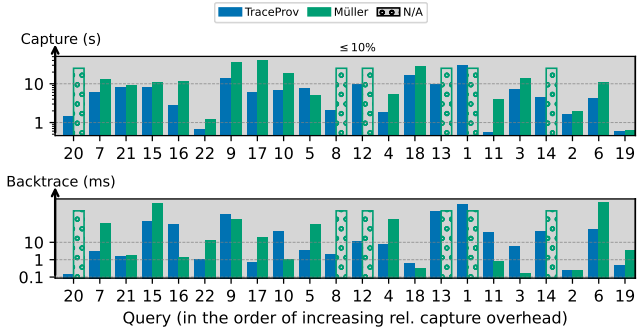


Figure 17: Logging and backtrace runtime of TraceProv and Müller for `P_single` on TPC-H SF10. Queries are ordered by relative capture overhead of TraceProv over NoProv.

$(id : \text{int}, z : \text{int}, v : \text{int})$ where z. is drawn from a Zipfian distribution ($\theta = 1$), and the number of distinct values of z is varied from 10, 100, to 1000. v is a a random value $\in [1, 100]$ sampled from a uniform distribution. We fix $|T| = 10M$. We compare against SmokedDuck for `P_single` for multiple threads (1, 4 and 12) in Figure 16.

For this query, TraceProv is, on average, 2.3× and 8.4× slower than SmokedDuck in capture and backtrace phase respectively. However, due to expensive index building, SmokedDuck has high setup cost (average of 317ms) which is, on average, 19× slower than SmokedDuck's capture query execution. For comparison, TraceProv pays only 102μs for setup, about 3100× faster than SmokedDuck. The total end-to-end results shown in Figure 16 are consistent with these observations. TraceProv has lower end-to-end time for deriving lineage of a single row.

7.6 Comparison against Müller

We now provide a detailed comparison against Müller for TPC-H SF=10 in Figure 17.

7.6.1 Logging. As indicated in Figure 17, the per-query capture overhead of TraceProv is always less than 10% over NoProv. Further, in most of the TPC-H queries, TraceProv has lesser capture than Müller. Indeed, *Q5* is the only query for which Müller has a slightly lesser capture overhead than TraceProv (2-3 seconds). The low capture overhead of TraceProv is due to Müller performing uniqueness checks during log inserts. TraceProv is upto 7.2x faster than Müller, and has median speedup of 2x over Müller.

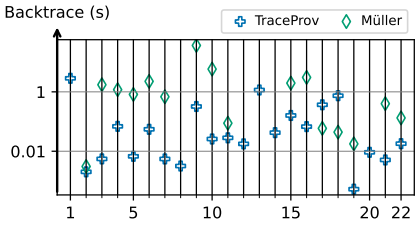


Figure 18: backtrace runtime of TraceProv and Müller for `P_all` on TPC-H SF10.

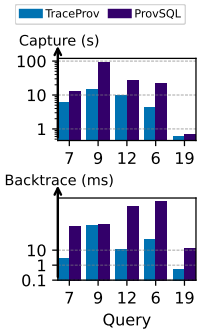


Figure 19: Logging and backtrace runtime of TraceProv and ProvSQL for `P_single` on TPC-H SF10. Queries are ordered by relative capture overhead of TraceProv over NoProv.

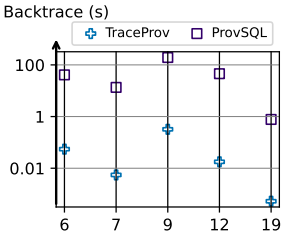


Figure 20: backtrace runtime of TraceProv and ProvSQL for `P_all` on TPC-H SF10.

7.6.2 Backtrace. In this phase, Müller has a notably better performance than TraceProv. This is because `P_single` queries for Müller allow PostgreSQL to utilize index scans (since the `P_single` become point queries). As we demonstrate later, TraceProv significantly outperforms Müller for `P_all` in backtrace phase.

7.6.3 Backtrace for `P_all`. Figure 18 show the backtrace time for TraceProv and Müller for TPC-H SF=10 `P_all`. As evident, TraceProv significantly outperforms Müller for most of the queries. Indeed, TraceProv is upto 314x faster than Müller and has a median speedup of 71x.

7.7 Comparison against ProvSQL

We now provide a detailed comparison against ProvSQL for TPC-H SF=10 in Figure 19.

7.7.1 Logging. As indicated in Figure 19 for all the TPC-H queries supported by ProvSQL, TraceProv has significantly lesser overhead than ProvSQL. TraceProv is upto 6.5x faster than ProvSQL and as median speedup of 3x over ProvSQL. This occurs since ProvSQL performs interprocess communication for log inserts.

7.7.2 Backtrace. For this phase, TraceProv is still significantly faster than ProvSQL. The maximum speedup of TraceProv is 739x and median speedup is 129x over ProvSQL. The ProvSQL requires multiple hash table lookups, while TraceProv often can perform backtrace for `P_single` via a single scan over the log files due to join-rewrite optimization.

7.7.3 Backtrace for `P_all`. Figure 20 show the backtrace time for TraceProv and Müller for TPC-H SF=10 `P_all`. As evident, TraceProv significantly outperforms ProvSQL for most of the queries. Indeed, TraceProv is upto 2576x faster than ProvSQL and has a median speedup of 1465x.

7.8 Provenance Polynomials

We now evaluate capturing provenance polynomials. TraceProv supports generation of provenance polynomials for SPJ queries. On PostgreSQL, we compare against ProvSQL and GProM. On DuckDB, we compare against SmokedDuck and GProM. We require all systems to output a string representation of the polynomial for every result tuple of a query. Both ProvSQL and TraceProv factorize polynomials based on the structure of the input query. For ProvSQL at every `DISTINCT` in the query, the system sums up the polynomials attached to all intermediate results with the same values, i.e., factorization can be controlled by adding duplicate elimination at strategic places in the query.

To stress-test a systems capability to exploit factorization, we use a query template shown below that produces a single result tuple (as a has a single distinct values in all input table). Here N controls the provenance size by determining how may rows from R_i are returned.

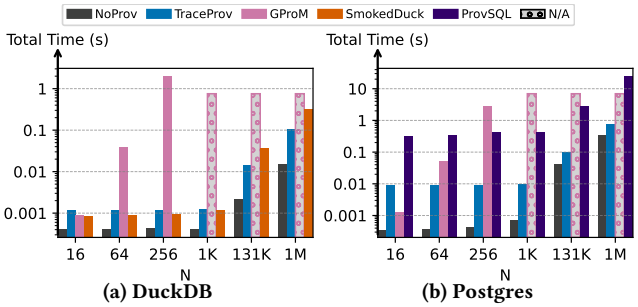
$$\delta(\Pi_a(\sigma_{b\leq N}(R_1))) \times \delta(\Pi_a((\sigma_{b\leq N}(R_2)))) \times \delta(\Pi_a(\sigma_{b\leq N}(R_3)))$$

Let r_i denote tuples from R_1, s_i denote tuples from R_2 and t_i denote tuples from R_3. TraceProv, SmokedDuck and ProvSQL produce a polynomial the form $(\sum_{i=1}^{N} r_i) \cdot (\sum_{j=1}^{N} s_j) \cdot (\sum_{k=1}^{N} t_k)$ whose size is linear in N. GProM, however, produces a polynomial of the form $\sum_{i,j,k}(r_i \cdot s_j \cdot t_k)$ whose size is $O(N^3)$. We present the execution time for capturing polynomials in Figure 21. As GProM does not utilize factorization, it does not scale to larger values of N, running out of memory. On DuckDB (Figure 21a), SmokedDuck initial outperforms TraceProv for smaller values of N. However, as N is increased to $1M$, TraceProv outperforms SmokedDuck. TraceProv consistently outperforms ProvSQL.

7.8.1 n-way joins. In this experiment we vary the number of joins in the query. Query $Q \bowtie n$ is an n-way join of the form:

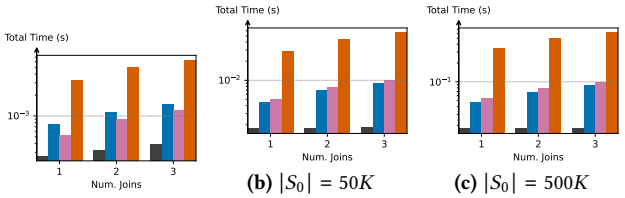
$$S_0 \bowtie_{S_0.z=S_1.id} S_1 \ldots S_{n-2} \bowtie_{S_{n-2}.id=S_{n-1}.id} S_{n-1}$$

We vary $|S_0| \in 5K, 50K, 500K$ to control z is drawn out of Zipfian distribution ($\theta = 1$). $|S_i| = 1,000 \ \forall i > 0$. Finally, for $S_{i>0}$, id is the primary key.



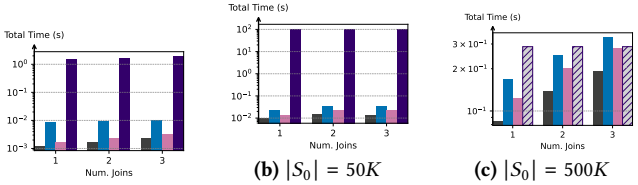
(a) DuckDB **(b) Postgres**

Figure 21: Runtime for capturing provenance polynomials.



(a) $|S_0| = 5K$ **(b)** $|S_0| = 50K$ **(c)** $|S_0| = 500K$

Figure 22: Runtimes for capturing provenance polynomials for multi-way joins on DuckDB.



(a) $|S_0| = 5K$ **(b)** $|S_0| = 50K$ **(c)** $|S_0| = 500K$

Figure 23: Runtime for capturing provenance polynomials for multi-way joins on Postgres

Table 1: Logging performance and log size per log record

		Number of Rows							
		PostgreSQL				DuckDB			
		10	1K	100K	10M	10	1K	100K	10M
Time μs	TraceProv					20	0.2	0.004	0.003
	TraceProv (file)	597	6.1	0.11	0.05	27	0.29	0.006	0.003
	Müller	86	5.0	4.1	4.1				
	ProvSQL	188	5.8	2.4	2.4				
	SmokedDuck					0.8	0.009	0.001	0.001
Size Bytes	TraceProv	1 K	16	8.0	8.0	3 K	32	4.3	4.0
	Müller	819	73	68	68				
	ProvSQL	2 M	25 K	469	350				
	SmokedDuck					7.2	4.0	4.0	4.0

7.9 Performance of Logging

One of the main enablers of high capture performance of TraceProv is its efficient parallel logging framework. We now evaluate logging performance in isolation. We now compare the cost of logging as the mean time and storage size per log record. We use a simple query $Q = \Pi_a(T)$ over relation $T(a)$ where a is a 4 byte integer. For this query, all rows in T belong in the provenance. We vary the number of records logged by varying $|T| \in \{10, 1K, 100K, 10M\}$. For ProvSQL, we explicitly call the function `create_gate` to create a gate per row. The results are shown in Table 1. TraceProv has the highest per-record time for fewer log records (10 and $1K$). However, as the number of log records is increased, the per-log time

Table 2: Backtrace optimizations: speed-up over unoptimized TraceProv for TPC-H SF10

	Query																					
Opt	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**	**15**	**16**	**17**	**18**	**19**	**20**	**21**	**22**
JR	0.99	0.77	2.1	1.1	1.4	0.88	1.3	1.1	1.5	2.0	1.2	1.1	1.0	0.97	1.1	1.1	1.0	1.0	2.8	2.8	1.0	1.2
Z	0.99	2.1	52	0.99	1.1	0.58	1.0	1.1	2.4	4.1	1.5	0.52	1.0	0.68	1.1	1.1	1.0	1.6	3.1	5.5	1.7	1.0
P	0.57	2.1	57	1.1	1.9	0.61	1.7	1.0	8.1	23	1.6	1.0	1.1	0.68	1.4	0.34	1.1	1.6	3.1	5.5	1.7	1.1

drops significantly. At $10M$ log records, TraceProv is 48× faster than ProvSQL, the next best system on PostgreSQL. For DuckDB, TraceProv benefits from vectorization for UDFs, resulting in a $\sim 20x$ speedup compared to PostgreSQL at 10M rows. While SmokedDuck is the fastest system in terms of raw logging performance, the absolute runtime difference per log record at 10M rows is only $2ns$. For PostgreSQL TraceProv uses 8 bytes to represent a log record. At fewer number of log records (10 and $1K$), the storage per log record is higher since TraceProv overrequests pages from the OS. Both Müller and ProvSQL have significantly higher storage requirements (8x and $\sim 43x$ at 10M rows). For DuckDB, TraceProv uses 4 bytes to represent log records. For increasing $|T|$, the system approaches the per record size of SmokedDuck which also uses 4 byte records. When using mmaped files, TraceProv shows slightly higher overhead when logging few records, but still approaches the $3ns$ per log record at 10M rows.

7.10 Backtrace Optimizations

As described in the Section 6, TraceProv implements multiple optimizations for the `P_single` task. Table 2 shows the speed-up for lineage capture backtrace over TraceProv with all optimizations deactivated. Each row applies all the optimizations above that row. *JR* (join-rewrite) is the optimization that eliminates the join with the top-most log if the lineage of a single result is requested, *Z* utilizes TraceProv's zone maps for data skipping, and *P* applies partition pruning. JR improves performance by 2x for several queries, even without use of zonemaps, at the cost of slight degradation for some queries. Activating both the join rewrites and zonemaps (Z) significantly improves the backtrace for most TPC-H queries e.g., 52x for *Q3* and 5.5x for *Q20*. Additionally applying partitioning (P) further improves the performance for *Q9* (8x) and *Q10* (23x). While the unoptimized backtrace has median time over all TPC-H queries of 2.75ms, activating all optimizations reduces this to 1.52ms. Note that applying partitioning causes DuckDB's query optimizer to switch from *Perfect Hash Join* to *Hash Join* in some cases which reduces performance for some queries.

8 CONCLUSIONS

In this paper, we present TraceProv, a novel bolt-on system for provenance capture in relational databases that solely relies on extensibility mechanisms to implement provenance support without compromising performance. In fact, TraceProv consistently outperforms SmokedDuck, the SOTA approach for provenance capture that extends the internals of DuckDB and our system significantly outperforms prior bolt-on approaches. This is made possible using a light-weight, parallel append-only logging framework and by combining propagation with logging in a principled manner. We demonstrating the generality and flexibility of our approach by implementing it for both a traditional row-based system (PostgreSQL) as well as in a column system (DuckDB). Interesting directions for future work include applying the system's logging infrastructure in other contexts, e.g., to capture provenance in distributed systems or data-intensive workflows, to support provenance capture for updates and transactions, and to add support for the general extension of provenance polynomials for aggregation from [2].

REFERENCES

[1] P. Agrawal, O. Benjelloun, A. D. Sarma, C. Hayworth, S. U. Nabar, T. Sugihara, and J. Widom. Trio: A system for data, uncertainty, and lineage. In *VLDB*, pages 1151–1154, 2006.

[2] Y. Amsterdamer, D. Deutch, and V. Tannen. Provenance for aggregate queries. In *PODS*, pages 153–164, 2011.

[3] B. Arab, S. Feng, B. Glavic, S. Lee, X. Niu, and Q. Zeng. Gprom - a swiss army knife for your provenance needs. *IEEE Data Eng. Bull.*, 41(1):51–62, 2018.

[4] P. Buneman, S. Khanna, and W.-C. Tan. Why and where: A characterization of data provenance. In *ICDT*, pages 316–330, 2001.

[5] P. Buneman, S. Khanna, and W.-C. Tan. On propagation of deletions and annotations through views. In *PODS*, pages 150–158, 2002.

[6] Y. Cui and J. Widom. Lineage tracing in a data warehousing system. In *ICDE*, pages 683–684, 2000.

[7] K. M. Dannert, E. Grädel, M. Naaf, and V. Tannen. Semiring provenance for fixed-point logic. In C. Baier and J. Goubault-Larrecq, editors, *29th EACSL Annual Conference on Computer Science Logic, CSL 2021, January 25-28, 2021, Ljubljana, Slovenia (Virtual Conference)*, volume 183, pages 17:1–17:22, 2021.

[8] D. Deutch, Z. Ives, T. Milo, and V. Tannen. Caravan: Provisioning for what-if analysis. *CIDR*, 2013.

[9] B. Dietrich, T. Müller, and T. Grust. Data provenance for recursive sql queries. In A. Chapman, D. Deutch, and T. Malik, editors, *TaPP*, pages 9:1–9:8, 2022.

[10] A. Esmailpourlialestani, B. Glavic, X. Hu, and S. Sintos. Smallest synthetic witnesses for conjunctive queries. *Proc. ACM Manag. Data*, 3(2):113:1–113:25, 2025.

[11] F. Geerts, G. Karvounarakis, V. Christophides, and I. Fundulaki. Algebraic structures for capturing the provenance of sparql queries. In *ICDT*, 2012.

[12] F. Geerts and A. Poggi. On database query languages for k-relations. *Journal of Applied Logic*, 8(2):173–185, 2010.

[13] B. Glavic. Data provenance - origins, applications, algorithms, and models. *Foundations and Trends® in Databases*, 9(3-4):209–441, 2021.

[14] B. Glavic and G. Alonso. Perm: Processing provenance and data on the same data model through query rewriting. In *ICDE*, pages 174–185, 2009.

[15] B. Glavic and G. Alonso. Provenance for nested subqueries. In *EDBT*, pages 982–993, 2009.

[16] B. Glavic, R. J. Miller, and G. Alonso. Using sql for efficient generation and querying of provenance information. *In search of elegance in the theory and practice of computation: a Festschrift in honour of Peter Buneman*, pages 291–320, 2013.

[17] T. J. Green, G. Karvounarakis, and V. Tannen. Provenance semirings. In *PODS*, pages 31–40, 2007.

[18] X. Hu and S. Sintos. Finding smallest witnesses for conjunctive queries. In *ICDT*, 2024.

[19] X. Hu, S. Sun, S. Patwa, D. Panigrahi, and S. Roy. Aggregated deletion propagation for counting conjunctive query answers. *PVLDB*, 14(2):228–240, 2020.

[20] V. Jha and B. Glavic. Traceprov: All you need is log - efficient bolt-on provenance capture for complex relational queries (extended version). https://github.com/realVinayak/traceprov/blob/main/techreport.pdf, 2026.

[21] A. Meliou and D. Suciu. Tiresias: The database oracle for how-to queries. In *SIGMOD*, pages 337–348, 2012.

[22] H. Mohammed, C. Summers, S. Kaushik, and E. Wu. Smokedduck demonstration: Sqlstepper. In S. Das, I. Pandis, K. S. Candan, and S. Amer-Yahia, editors, *SIGMOD*, pages 183–186, 2023.

[23] H. Mohammed and E. Wu. Lineage capture trade-offs: A case study in duckdb. In *Proceedings of the ProvenanceWeek 2025*, pages 32–36. 2025.

[24] H. Mohammed, E. Wu, A. Yao, C. Summers, L. Flokas, G. Y.-Y. Chan, S. Mitra, and H. Zhong. Fade: More than a million what-ifs per second. *PVLDB*, 18(4):943–955, 2024.

[25] M. Mutsuzaki, M. Theobald, A. de Keijzer, J. Widom, P. Agrawal, O. Benjelloun, A. Das Sarma, R. Murthy, and T. Sugihara. Trio-one: Layering uncertainty and lineage on a conventional dbms. In *CIDR*, pages 269–274, 2007.

[26] T. Müller, B. Dietrich, and T. Grust. You say'what', i hear'where'and'why':(mis-) interpreting sql to derive fine-grained provenance. *PVLDB*, 11(11):1536–1549, 2018.

[27] T. Müller and T. Grust. Provenance for sql through abstract interpretation: Value-less, but worthwhile. *PVLDB*, 8(12), 2015.

[28] X. Niu, R. Kapoor, B. Glavic, D. Gawlick, Z. H. Liu, V. Krishnaswamy, and V. Radhakrishnan. Provenance-aware query optimization. In *ICDE*, pages 473–484,

2017.
[29] X. Niu, R. Kapoor, B. Glavic, D. Gawlick, Z. H. Liu, V. Krishnaswamy, and V. Radhakrishnan. Heuristic and cost-based optimization for diverse provenance tasks. *TKDE*, 2018.
[30] D. Olteanu. Recent increments in incremental view maintenance. In F. Geerts and W. Martens, editors, *PODS*, pages 8–17, 2024.
[31] P. Pintor, R. L. de C. Costa, and J. M. Moreira. A dbms-independent approach for capturing provenance polynomials through query rewriting. *IEEE Access*, 14:66114–66133, 2026.
[32] F. Psallidas and E. Wu. Smoke: Fine-grained lineage at interactive speed. *PVLDB*, 11(6):719–732, 2018.
[33] A. Sen, S. Maniu, and P. Senellart. Provsql: A general system for keeping track of the provenance and probability of data. In *2026 IEEE 42nd International Conference on Data Engineering (ICDE)*. IEEE, 2026.
[34] P. Senellart, L. Jachiet, S. Maniu, and Y. Ramusat. Provsql: provenance and probability management in postgresql. *PVLDB*, 11(12):2034–2037, 2018.
[35] Transaction Processing Council. TPC-H Benchmark Specification, 2009.
[36] G. Van den Broeck and D. Suciu. Query processing on probabilistic data: A survey. 2017.